C++ Chapter 01 Notes

Chapter 01 — Notes

Chapter Review

A **statement** is a type of instruction that causes the program to perform some action. Statements are often terminated by a semicolon.

A **function** is a collection of statements that execute sequentially. Every C++ program must include a special function named *main*. When you run your program, execution starts at the top of the *main* function.

In programming, the name of a function (or object, type, template, etc…) is called its **identifier**.

The rules that govern how elements of the C++ language are constructed is called **syntax**. A **syntax error** occurs when you violate the grammatical rules of the language.

Comments allow the programmer to leave notes in the code. C++ supports two types of comments. Line comments start with a `//` and run to the end of the line. Block comments start with a `/*` and go to the paired `*/` symbol. Don't nest block comments.

You can use comments to temporarily disable lines or sections of code. This is called commenting out your code.

Data is any information that can be moved, processed, or stored by a computer. A single piece of data is called a **value**. Common examples of values include letters (e.g. `a`), numbers (e.g. `5`), and text (e.g. `Hello`).

A variable is a named piece of memory that we can use to store values. In order to create a variable, we use a statement called a **definition statement**. When the program is run, each defined variable is **instantiated**, which means it is assigned a memory address.

A **data type** tells the compiler how to interpret a piece of data into a meaningful value. An **integer** is a number that can be written without a fractional component, such as 4, 27, 0, -2, or -12.

Copy assignment (via operator=) can be used to assign an already created variable a value.

The process of specifying an initial value for an object is called **initialization**, and the syntax used to initialize an object is called an **initializer**.

Simplified, C++ supports 6 basic types of initialization:

Initialization Type	Example	Note
Default-initialization	int x;	In most cases, leaves variable with indeterminate value
Copy-initialization	int x = 5;	
Direct-initialization	int x (5);	
Direct-list-initialization	int x { 5 };	Narrowing conversions disallowed
Copy-list-initialization	int x = { 5 };	Narrowing conversions disallowed
Value-initialization	int x {};	Usually performs zero-initialization

Direct-initialization is sometimes called parenthesis-initialization, and list-initialization (including value-initialization) is sometimes called uniform-initialization or brace-initialization. You should prefer brace-initialization over the other initialization forms, and prefer initialization over assignment.

Although you can define multiple variables in a single statement, it's better to define and initialize each variable on its own line, in a separate statement.

std::cout and `operator<<` allow us to output the result of an expression to the console.

std::endl outputs a newline character, forcing the console cursor to move to the next line, and flushes any pending output to the console. The `'\n'` character also outputs a newline character, but lets the system decide when to flush the output. Be careful not to use `'/n'` (forward slash).

std::cin and `operator>>` allow us to get a value from the keyboard.

A variable that has not been given a value is called an **uninitialized variable**. Trying to get the value of an uninitialized variable will result in **undefined behavior**, which can manifest in any number of ways.

C++ reserves a set of names called **keywords**. These have special meaning within the language and may not be used as variable names.

A **literal constant** is a fixed value inserted directly into the source code. Examples are `5` and `"Hello world!"`.

An **operation** is a process involving zero or more input values, called **operands**. The specific operation to be performed is denoted by the provided **operator**. The result of an operation produces an output value.

Unary operators take one operand. **Binary** operators take two operands, often called left and right. **Ternary** operators take three operands. **Nullary** operators take zero operands.

An **expression** is a sequence of literals, variables, operators, and function calls that are evaluated to produce a single output value. The calculation of this output value is called **evaluation**. The value produced is the **result** of the expression.

An **expression statement** is an expression that has been turned into a statement by placing a semicolon at the end of the expression.

When writing programs, add a few lines or a function, compile, resolve any errors, and make sure it works. Don't wait until you've written an entire program before compiling it for the first time!

Focus on getting your code working. Once you are sure you are going to keep some bit of code, then you can spend time removing (or commenting out) temporary/debugging code, adding comments, handling error cases, formatting your code, ensuring best practices are followed, removing redundant logic, etc…

First-draft programs are often messy and imperfect. Most code requires cleanup and refinement to get to great!

Quiz time

What is the difference between initialization and assignment? How many times can a variable be initialized or assigned a value?

> **Solution**
>
> Initialization provides a variable with an initial value (at the point of creation). Assignment gives a variable a new value after the variable has already been defined.
>
> Since a variable is only created once, it can only be initialized once. A variable can be assigned a value as many times as desired.

When does undefined behavior occur? What are the consequences of undefined behavior?

> **Solution**
>
> Undefined behavior occurs when the programmer does something that is ill-specified by the C++ language. The consequences could be almost anything, from crashing to producing the wrong answer to working correctly anyway.

Write a program that asks the user to enter a number, and then enter a second number. The program should tell the user what the result of adding and subtracting the two numbers is.

The output of the program should match the following (assuming inputs of 6 and 4):

```
Enter an integer: 6
Enter another integer: 4
6 + 4 is 10.
6 - 4 is 2.
```

Hint: To print a period and a newline, use `".\n"`, not `'.\n'`.

> **Solution**

```cpp
#include <iostream>

int main()
{
  std::cout << "Enter an integer: ";
  int x{};
  std::cin >> x;

  std::cout << "Enter another integer: ";
  int y{};
  std::cin >> y;

  std::cout << x << " + " << y << " is " << x + y << ".\n";
  std::cout << x << " - " << y << " is " << x - y << ".\n";

  return 0;
}
```

Comments

A **comment** is a programmer-readable note that is inserted directly into the source code of the program. Comments are ignored by the compiler and are for the programmer's use only.

In C++ there are two different styles of comments, both of which serve the same purpose: to help programmers document the code in some way.

Single-line comments

The `//` symbol begins a C++ single-line comment, which tells the compiler to ignore everything from the `//` symbol to the end of the line. For example:

```
std::cout << "Hello world!"; // Everything from here to the end of the line is ignored
```

Typically, the single-line comment is used to make a quick comment about a single line of code.

```
std::cout << "Hello world!\n"; // std::cout lives in the iostream library
std::cout << "It is very nice to meet you!\n"; // these comments make the code hard to read
std::cout << "Yeah!\n"; // especially when lines are different lengths
```

Having comments to the right of a line can make both the code and the comment hard to read, particularly if the line is long. If the lines are fairly short, the comments can simply be aligned (usually to a tab stop), like so:

```
std::cout << "Hello world!\n";                // std::cout lives in the iostream library
std::cout << "It is very nice to meet you!\n"; // this is much easier to read
std::cout << "Yeah!\n";                       // don't you think so?
```

However, if the lines are long, placing comments to the right can make your lines really long. In that case, single-line comments are often placed above the line it is commenting:

```
// std::cout lives in the iostream library
std::cout << "Hello world!\n";

// this is much easier to read
std::cout << "It is very nice to meet you!\n";

// don't you think so?
std::cout << "Yeah!\n";
```

> **Author's note**
>
> In this tutorial series, our examples fall into one of the following categories:
>
> - Full programs (those with a `main()` function). These are ready to be compiled and run.
> - Snippets (small pieces) of code, such as the statements above. We use these to demonstrate specific concepts in a concise manner.
>
> We don't intend for you to compile snippets. But if you'd like to, you'll need to turn them into a full program. Typically, that program will look something like this:

```
#include <iostream>

int main()
{
    // Replace this line with the snippet(s) of code you'd like to compile

    return 0;
}
```

Multi-line comments

The `/*` and `*/` pair of symbols denotes a C-style multi-line comment. Everything in between the symbols is ignored.

```
/* This is a multi-line comment.
   This line will be ignored.
   So will this one. */
```

Since everything between the symbols is ignored, you will sometimes see programmers “beautify” their multi-line comments:

```
/* This is a multi-line comment.
 * the matching asterisks to the left
 * can make this easier to read
 */
```

Multi-line style comments can not be nested. Consequently, the following will have unexpected results:

```
/* This is a multi-line /* comment */ this is not inside the comment */
// The above comment ends at the first */, not the second */
```

When the compiler tries to compile this, it will ignore everything from the first `/*` to the first `*/`. Since `this is not inside the comment */` is not considered part of the comment, the compiler will try to compile it. That will inevitably result in a compile error.

This is one place where using a syntax highlighter can be really useful, as the different coloring for comment should make clear what’s considered part of the comment vs not.

> **Warning**
>
> Don’t use multi-line comments inside other multi-line comments. Wrapping single-line comments inside a multi-line comment is okay.

Proper use of comments

Typically, comments should be used for three things. First, for a given library, program, or function, comments are best used to describe *what* the library, program, or function, does. These are typically placed at the top of the file or library, or immediately preceding the function. For example:

```
// This program calculates the student's final grade based on their test and homework scores.
```

```
// This function uses Newton's method to approximate the root of a given equation.
```

```
// The following lines generate a random item based on rarity, level, and a weight factor.
```

All of these comments give the reader a good idea of what the library, program, or function is trying to accomplish without having to look at the actual code. The user (possibly someone else, or you if you're trying to reuse code you've previously written) can tell at a glance whether the code is relevant to what he or she is trying to accomplish. This is particularly important when working as part of a team, where not everybody will be familiar with all of the code.

Second, *within* a library, program, or function described above, comments can be used to describe *how* the code is going to accomplish its goal.

```
/* To calculate the final grade, we sum all the weighted midterm and homework scores
    and then divide by the number of scores to assign a percentage, which is
    used to calculate a letter grade. */
```

```
// To generate a random item, we're going to do the following:
// 1) Put all of the items of the desired rarity on a list
// 2) Calculate a probability for each item based on level and weight factor
// 3) Choose a random number
// 4) Figure out which item that random number corresponds to
// 5) Return the appropriate item
```

These comments give the user an idea of how the code is going to accomplish its goal without having to understand what each individual line of code does.

Third, at the statement level, comments should be used to describe *why* the code is doing something. A bad statement comment explains *what* the code is doing. If you ever write code that is so complex that needs a comment to explain *what* a statement is doing, you probably need to rewrite your statement, not comment it.

Here are some examples of bad line comments and good statement comments.

Bad comment:

```
// Set sight range to 0
sight = 0;
```

Reason: We already can see that sight is being set to 0 by looking at the statement

Good comment:

```
// The player just drank a potion of blindness and can not see anything
sight = 0;
```

Reason: Now we know why the player's sight is being set to 0

Bad comment:

```
// Calculate the cost of the items
cost = quantity * 2 * storePrice;
```

Reason: We can see that this is a cost calculation, but why is quantity multiplied by 2?

Good comment:

```
// We need to multiply quantity by 2 here because they are bought in pairs
cost = quantity * 2 * storePrice;
```

Reason: Now we know why this formula makes sense.

Programmers often have to make a tough decision between solving a problem one way, or solving it another way. Comments are a great way to remind yourself (or tell somebody else) the reason you made one decision instead of another.

Good comments:

```
// We decided to use a linked list instead of an array because
// arrays do insertion too slowly.
```

```
// We're going to use Newton's method to find the root of a number because
// there is no deterministic way to solve these equations.
```

Finally, comments should be written in a way that makes sense to someone who has no idea what the code does. It is often the case that a programmer will say "It's obvious what this does! There's no way I'll forget about this". Guess what? It's *not* obvious, and you *will* be amazed how quickly you forget. :) You (or someone else) will thank you later for writing down the what, how, and why of your code in human language. Reading individual lines of code is easy. Understanding what goal they are meant to accomplish is not.

> **Related content**
>
> We discuss commenting for variable declaration statements in lesson 1.7 – Keywords and naming identifiers.

> **Best practice**
>
> Comment your code liberally, and write your comments as if speaking to someone who has no idea what the code does. Don't assume you'll remember why you made specific choices.

> **Author's note**
>
> Throughout the rest of this tutorial series, we'll use comments inside code blocks to draw your attention to specific things, or help illustrate how things work (while ensuring the programs still compile). Astute readers will note that by the above standards, most of these comments are horrible. :) As you read through the rest of the tutorials, keep in mind that the comments are serving an intentional educational purpose, not trying to demonstrate what good comments look like.

> **As an aside…**
>
> Documentation generation programs such as Doxygen are designed to help generate and leverage documentation in various ways. Amongst other things, they can:
>
> - Help standardize the way your code is documented.
> - Generate diagrams, visualizations, and cross-links to make understanding the code structure easier.
> - Export your documentation to other formats (e.g. HTML) so it can be easily shared with others (e.g. other team members, or developers who are integrating whatever you are writing).
>
> You won't need these while learning the language, but you may encounter them or find them useful in the future, especially in professional environments.

Commenting out code

Converting one or more lines of code into a comment is called **commenting out** your code. This provides a convenient way to (temporarily) exclude parts of your code from being included in your compiled program.

To comment out a single line of code, simply use the `//` style comment to turn a line of code into a comment temporarily:

Uncommented out:

```
std::cout << 1;
```

Commented out:

```
//    std::cout << 1;
```

To comment out a block of code, use `//` on multiple lines of code, or the `/* */` style comment to turn the block of code into a comment temporarily.

Uncommented out:

```
std::cout << 1;
    std::cout << 2;
    std::cout << 3;
```

Commented out:

```
//    std::cout << 1;
//    std::cout << 2;
//    std::cout << 3;
```

or

```
/*
    std::cout << 1;
    std::cout << 2;
    std::cout << 3;
*/
```

There are quite a few reasons you might want to do this:

1. You're working on a new piece of code that won't compile yet, and you need to run the program. The compiler won't let you compile the code if there are compiler errors. Commenting out the code that won't compile will allow the program to compile so you can run it. When you're ready, you can uncomment the code, and continue working on it.
2. You've written new code that compiles but doesn't work correctly, and you don't have time to fix it until later. Commenting out the broken code will ensure the broken code doesn't execute and cause problems until you can fix it.
3. To find the source of an error. If a program isn't producing the desired results (or is crashing), it can sometimes be useful to disable parts of your code to see if you can isolate what's causing it to not work correctly. If you comment out one or more lines of code, and your program starts working as expected (or stops crashing), odds are whatever you last commented out was part of the problem. You can then investigate why those lines of code are causing the problem.
4. You want to replace one piece of code with another piece of code. Instead of just deleting the original code, you can comment it out and leave it there for reference until you're sure your new code works properly. Once you are sure your new code is working, you can remove the old commented out code. If you can't get your new code to work, you can always delete the new code and uncomment the old code to revert to what you had before.

Commenting out code is a common thing to do while developing, so many IDEs provide support for commenting out a highlighted section of code. How you access this functionality varies by IDE.

> **For Visual Studio users**
>
> You can comment or uncomment a selection via Edit menu > Advanced > Comment Selection (or Uncomment Selection).

> **For Code::Blocks users**

> You can comment or uncomment a selection via Edit menu > Comment (or Uncomment, or Toggle comment, or any of the other comment tools).

> **For VS Code users**
>
> You can comment or uncomment a selection by pressing ctrl-/.

> **Tip**
>
> If you always use single line comments for your normal comments, then you can always use multi-line comments to comment out your code without conflict. If you use multi-line comments to document your code, then commenting-out code using comments can become more challenging.
>
> If you do need to comment out a code block that contains multi-line comments, you can also consider using the `#if 0` preprocessor directive, which we discuss in lesson 2.10 – Introduction to the preprocessor.

Summary

- At the library, program, or function level, use comments to describe *what*.
- Inside the library, program, or function, use comments to describe *how*.
- At the statement level, use comments to describe *why*.

Developing Your First Program

The preceding lessons have introduced a lot of terminology and concepts that we'll use in just about every program we create. In this lesson, we'll walk through the process of integrating this knowledge into our first simple program.

Multiply by 2

First, let's create a program that asks the user to enter an integer, waits for them to input an integer, then tells them what 2 times that number is. The program should produce the following output (assume I entered 4 as input):

```
Enter an integer: 4
Double that number is: 8
```

How do we tackle this? In steps.

> **Best practice**
>
> New programmers often try to write an entire program all at once, and then get overwhelmed when it produces a lot of errors. A better strategy is to add one piece at a time, make sure it compiles, and test it. Then when you're sure it's working, move on to the next piece.

We'll leverage that strategy here. As we go through each step, type (don't copy/paste) each program into your code editor, compile, and run it.

First, create a new console project.

Now let's start with some basic scaffolding. We know we're going to need a main() function (since all C++ programs must have one), so if your IDE didn't create a blank one when you created a new project, let's create one:

```
int main()
{
    return 0;
}
```

We know we're going to need to output text to the console, and get text from the user's keyboard, so we need to include iostream for access to std::cout and std::cin.

```
#include <iostream>

int main()
{
    return 0;
}
```

Now let's tell the user that we need them to enter an integer:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";

    return 0;
}
```

At this point, your program should produce this result:

```
Enter an integer:
```

and then terminate.

Next, we're going to get the user's input. We'll use std::cin and `operator>>` to get the user's input. But we also need to define a variable to store that input for use later.

```
#include <iostream>

int main() // note: this program has an error somewhere
{
    std::cout << "Enter an integer: ";

    int num{ }; // define variable num as an integer variable
    std::cin << num; // get integer value from user's keyboard

    return 0;
}
```

Time to compile our changes... and...

Uh oh! Here's what the author got on Visual Studio 2017:

```
1>------ Build started: Project: Double, Configuration: Release Win32 ------
1>Double.cpp
1>c:\vcprojects\double\double.cpp(8): error C2678: binary '<<': no operator found which takes a left-hand o
of type 'std::istream' (or there is no acceptable conversion)
1>c:\vcprojects\double\double.cpp: note: could be 'built-in C++ operator<<(bool, int)'
1>c:\vcprojects\double\double.cpp: note: while trying to match the argument list '(std::istream, int)'
1>Done building project "Double.vcxproj" -- FAILED.
========== Build: 0 succeeded, 1 failed, 0 up-to-date, 0 skipped ==========
```

We ran into a compile error!

First, since the program compiled before we made this latest update, and doesn't compile now, the error *must* be in the code we just added (lines 7 and 8). That significantly reduces the amount of code we have to scan to find the error. Line 7 is pretty straightforward (just a variable definition), so the error probably isn't there. That leaves line 8 as the likely culprit.

Second, this error message isn't very easy to read. But let's pick apart some key elements: The compiler is telling us it ran into the error on line 8. That means the actual error is probably on line 8, or possibly the preceding line, which reinforces our previous assessment. Next, the compiler is telling you that it couldn't find a '<<' operator that has a left-hand operand of type std::istream (which is the type of std::cin). Put another way, operator<< doesn't know what to do with std::cin, so the error must be either with our use of std::cin or our use of operator<<.

See the error now? If you don't, take a moment and see if you can find it.

Here's the program that contains the corrected code:

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";

    int num{ };
    std::cin >> num; // std::cin uses operator >>, not operator <<!

    return 0;
}
```

Now the program will compile, and we can test it. The program will wait for you to enter a number, so let's enter 4. The output should look like this:

```
Enter an integer: 4
```

Almost there! Last step is to double the number.

Once we finish this last step, our program will compile and run successfully, producing the desired output.

There are (at least) 3 ways we can go about this. Let's go from worst to best.

The not-good solution

```
#include <iostream>

// worst version
int main()
{
    std::cout << "Enter an integer: ";

    int num{ };
    std::cin >> num;

    num = num * 2; // double num's value, then assign that value back to num

    std::cout << "Double that number is: " << num << '\n';

    return 0;
}
```

In this solution, we use an expression to multiply *num* by 2, and then assign that value back to *num*. From that point forward, *num* will contain our doubled number.

Why this is a bad solution:

- Before the assignment statement, num contains the user's input. After the assignment, it contains a different value. That's confusing.
- We overwrote the user's input by assigning a new value to the input variable, so if we wanted to extend our program to do something else with that input value later (e.g. triple the user's input), it's already been lost.

The mostly-good solution

```
#include <iostream>

// less-bad version
int main()
{
    std::cout << "Enter an integer: ";

    int num{ };
    std::cin >> num;

    int doublenum{ num * 2 }; // define a new variable and initialize it with num * 2
    std::cout << "Double that number is: " << doublenum << '\n'; // then print the value of that variable h

    return 0;
}
```

This solution is pretty straightforward to read and understand, and resolves both of the problems encountered in the worst solution.

The primary downside here is that we're defining a new variable (which adds complexity) to store a value we only use once. We can do better.

The preferred solution

```
#include <iostream>

// preferred version
int main()
{
    std::cout << "Enter an integer: ";

    int num{ };
    std::cin >> num;

    std::cout << "Double that number is: " <<  num * 2 << '\n'; // use an expression to multiply num * 2 at
point where we are going to print it

    return 0;
}
```

This is the preferred solution of the bunch. When std::cout executes, the expression *num* 2* will get evaluated, and the result will be double *num*'s value. That value will get printed. The value in *num* itself will not be altered, so we can use it again later if we wish.

This version is our reference solution.

> **Author's note**
>
> The first and primary goal of programming is to make your program work. A program that doesn't work isn't useful regardless of how well it's written.
>
> However, there's a saying I'm fond of: "You have to write a program once to know how you should have written it the first time." This speaks to the fact that the best solution often isn't obvious, and that our first solutions to problems are usually not as good as they could be.
>
> When we're focused on figuring out how to make our programs work, it doesn't make a lot of sense to invest a lot of time into code we don't even know if we'll keep. So we take shortcuts. We skip things like error handling and comments. We sprinkle debugging code

throughout our solution to help us diagnose issues and find errors. We learn as we go – things we thought might work don't work after all, and we have to backtrack and try another approach.

The end result is that our initial solutions often aren't well structured, robust (error-proof), readable, or concise. So once your program is working, your job really isn't done (unless the program is a one-off/throwaway). The next step is to cleanup your code. This involves things like: removing (or commenting out) temporary/debugging code, adding comments, handling error cases, formatting your code, and ensuring best practices are followed. And even then, your program may not be as simple as it could be – perhaps there is redundant logic that can be consolidated, or multiple statements that can be combined, or variables that aren't needed, or a thousand other little things that could be simplified. Too often new programmers focus on optimizing for performance when they should be optimizing for maintainability.

Very few of the solutions presented in these tutorials came out great the first time. Rather, they're the result of continual refinement until nothing else could be found to improve. And in many cases, readers still find plenty of other things to suggest as improvements!

All of this is really to say: don't be frustrated if/when your solutions don't come out wonderfully optimized right out of your brain. That's normal. Perfection in programming is an iterative process (one requiring repeated passes).

Author's note

One more thing: You may be thinking, "C++ has so many rules and concepts. How do I remember all of this stuff?".

Short answer: You don't. C++ is one part using what you know, and two parts looking up how to do the rest.

As you read through this site for the first time, focus less on memorizing specifics, and more on understanding what's possible. Then, when you have a need to implement something in a program you're writing, you can come back here (or to a reference site) and refresh yourself on how to do so.

Quiz time

Question #1

Modify the solution to the "best solution" program above so that it outputs like this (assuming user input 4):

```
Enter an integer: 4
Double 4 is: 8
Triple 4 is: 12
```

```
#include <iostream>

int main()
{
    std::cout << "Enter an integer: ";

    int num{ };
    std::cin >> num;

    std::cout << "Double " << num << " is: " << num * 2 << '\n';
    std::cout << "Triple " << num << " is: " << num * 3 << '\n';

    return 0;
}
```

Introduction to Expressions

Expressions

Consider the following series of statements, each of which defines a variable and initializes it:

```
// five() is a function that returns the value 5
int five()
{
    return 5;
}

int main()
{
    int a{ 2 };             // initialize variable a with literal value 2
    int b{ 2 + 3 };         // initialize variable b with computed value 5
    int c{ (2 * 3) + 4 };   // initialize variable c with computed value 10
    int d{ b };             // initialize variable d with variable value 5
    int e{ five() };        // initialize variable e with function return value 5

    return 0;
}
```

Note that the initializers above make use of a variety of different entities: literals, variables, operators, and function calls. Somehow, C++ is converting all of these different things into a single value that can then be used as the initial value for the variable.

What do all of these initializers have in common? They make use of an expression.

In general programming, an **expression** is a non-empty sequence of literals, variables, operators, and function calls that calculates a value. The process of executing an expression is called **evaluation**, and the resulting value produced is called the **result** of the expression (also sometimes called the **return value**).

> **For advanced readers**
>
> In C++, the result of an expression is one of the following:
>
> - A value (most commonly)
> - An object or a function. We discuss expressions that return objects in lesson 12.2 – Value categories (lvalues and rvalues).
> - Nothing. These are the result of non-value returning function calls (covered in lesson 2.3 – Void functions (non-value returning functions)) that are called only for their side effects
>
> For now, to keep things simple, we'll assume expressions are evaluated to produce values.

When an expression is evaluated, each of the terms inside the expression are evaluated, until a single value remains. Here are some examples of different kinds of expressions, with comments indicating how they evaluate:

```
2               // 2 is a literal that evaluates to value 2
"Hello world!"  // "Hello world!" is a literal that evaluates to text "Hello world!"
x               // x is a variable that evaluates to the value held by variable x
2 + 3           // operator+ uses operands 2 and 3 to evaluate to value 5
five()          // evaluates to the return value of function five()
```

As you can see, literals evaluate to their own values. Variables evaluate to the value of the variable. Operators (such as `operator+`) use their operands to evaluate to some other value. We haven't covered function calls yet, but in the context of an expression, function calls evaluate

to whatever value the function returns.

> **For advanced readers**
>
> Expressions involving operators with side effects are a little more tricky:
>
> ```
> x = 5           // x = 5 has side effect of assigning 5 to x, evaluates to x
> x = 2 + 3       // has side effect of assigning 5 to x, evaluates to x
> std::cout << x  // has side effect of printing value of x to console, evaluates to std::cout
> ```

> **Key insight**
>
> Wherever a single value is expected in C++, you can use a value-producing expression instead, and the expression will be evaluated to produce a single value.

Expressions do not end in a semicolon, and cannot be compiled by themselves. For example, if you were to try compiling the expression `x = 5`, your compiler would complain (probably about a missing semicolon). Rather, expressions are always evaluated as part of statements.

For example, take this statement:

```
int x{ 2 + 3 }; // 2 + 3 is an expression that has no semicolon -- the semicolon is at the end of the state
containing the expression
```

If you were to break this statement down into its syntax, it would look like this:

`type identifier { expression };`

type could be any valid type (we chose `int`). *identifier* could be any valid name (we chose `x`). And *expression* could be any valid expression (we chose `2 + 3`, which uses two literals and an operator).

Expression statements

Certain expressions (such as `x = 5`) are used primarily for their side effects (in this case, to assign the value `5` to the variable `x`) rather than the value they produce.

> **Related content**
>
> We cover side effects in lesson 1.9 – Introduction to literals and operators).

However, we mentioned above that expressions cannot be executed by themselves – they must exist as part of a statement. Fortunately, it's trivial to convert any expression into an equivalent statement. An **expression statement** is a statement that consists of an expression followed by a semicolon. When the expression statement is executed, the expression will be evaluated.

Thus, we can take any expression (such as `x = 5`), and turn it into an expression statement (`x = 5;`) that will compile.

When an expression is used in an expression statement, any result generated by the expression is discarded (because it is not used). For example, when the expression `x = 5` evaluates, the return value of `operator=` is discarded. And that's fine, because we just wanted to assign `5` to `x` anyway.

Useless expression statements

We can also make expression statements that compile but have no effect. For example, the expression statement (`2 * 3;`) is an expression statement whose expression evaluates to the result value of *6*, which is then discarded. While syntactically valid, such expression statements are useless. Some compilers (such as gcc and Clang) will produce warnings if they can detect that an expression statement is useless.

Subexpressions, full expressions, and compound expressions

We occasionally need to talk about specific kinds of expressions. For this purpose, we will define some related terms.

Consider the following expressions:

```
2               // 2 is a literal that evaluates to value 2
2 + 3           // 2 + 3 uses operator+ to evaluate to value 5
x = 4 + 5       // 4 + 5 evaluates to value 9, which is then assigned to variable x
```

Simplifying a bit, a **subexpression** is an expression used as an operand. For example, the subexpressions of `x = 4 + 5` are `x` and `4 + 5`. The subexpressions of `4 + 5` are `4` and `5`.

A **full expression** is an expression that is not a subexpression. All three expressions above (`2`, `2 + 3`, and `x = 4 + 5`) are full expressions.

In casual language, a **compound expression** is an expression that contains two or more uses of operators. `x = 4 + 5` is a compound expression because it contains two uses of operators (`operator=` and `operator+`). `2` and `2 + 3` are not compound expressions.

Quiz time

What is the difference between a statement and an expression?

> **Show Solution**
>
> Statements are used when we want the program to perform an action. Expressions are used when we want the program to calculate a value.

Indicate whether each of the following lines are *statements that do not contain expressions*, *statements that contain expressions*, or are *expression statements*.

a.

```
int x;
```

> **Show Solution**
>
> Statement does not contain an expression. This is a variable definition statement. `x` is not an expression in this context because it is being defined, not evaluated.

b.

```
int x = 5;
```

> **Show Solution**
>
> Statement contains an expression. `int x` is a variable definition. The `=` is part of the syntax for copy initialization. The initializer to the right of the equals sign is an expression.

c.

```
x = 5;
```

> **Show Solution**
>
> Expression statement. `x = 5` is a call to `operator=` with two operands: `x` and `5`. The semicolon makes it an expression statement.

d. Extra credit:

```
foo(); // foo is a function
```

Show Solution

Function calls are part of an expression, so this is an expression statement.

e. Extra credit:

```
std::cout << x; // Hint: operator<< is a binary operator.
```

Show Solution

`operator<<` is a binary operator, so `std::cout` must be the left-hand operand, and `x` must be the right-hand operand. Since that's the entire statement, this must be an expression statement.

Determine what values the following program outputs. Do not compile this program. Just work through it line by line in your head.

```
#include <iostream>

int main()
{
    std::cout << 2 + 3 << '\n';

    int x{ 6 };
    int y{ x - 2 };
    std::cout << y << '\n';

    int z{};
    z = x;
    std::cout << z * x << '\n';

    return 0;
}
```

Show Solution

```
5
4
36
```

Introduction to iostream: cout, cin, and endl

In this lesson, we'll talk more about `std::cout`, which we used in our *Hello world!* program to output the text *Hello world!* to the console. We'll also explore how to get input from the user, which we will use to make our programs more interactive.

The input/output library

The **input/output library** (io library) is part of the C++ standard library that deals with basic input and output. We'll use the functionality in this library to get input from the keyboard and output data to the console. The *io* part of *iostream* stands for *input/output*.

To use the functionality defined within the *iostream* library, we need to include the *iostream* header at the top of any code file that uses the content defined in *iostream*, like so:

```
#include <iostream>

// rest of code that uses iostream functionality here
```

std::cout

The *iostream* library contains a few predefined variables for us to use. One of the most useful is **std::cout**, which allows us to send data to the console to be printed as text. *cout* stands for "character output".

As a reminder, here's our *Hello world* program:

```
#include <iostream> // for std::cout

int main()
{
    std::cout << "Hello world!"; // print Hello world! to console

    return 0;
}
```

In this program, we have included *iostream* so that we have access to *std::cout*. Inside our *main* function, we use *std::cout*, along with the **insertion operator (`<<`)**, to send the text *Hello world!* to the console to be printed.

std::cout can not only print text, it can also print numbers:

```
#include <iostream> // for std::cout

int main()
{
    std::cout << 4; // print 4 to console

    return 0;
}
```

This produces the result:

```
4
```

It can also be used to print the value of variables:

```
#include <iostream> // for std::cout

int main()
{
    int x{ 5 }; // define integer variable x, initialized with value 5
    std::cout << x; // print value of x (5) to console
    return 0;
}
```

This produces the result:

```
5
```

To print more than one thing on the same line, the insertion operator (`<<`) can be used multiple times in a single statement to concatenate (link together) multiple pieces of output. For example:

```
#include <iostream> // for std::cout

int main()
{
    std::cout << "Hello" << " world!";
    return 0;
}
```

This uses the `<<` operator twice, first to output `Hello` and then to output `world`.

Thus, this program prints:

```
Hello world!
```

> **Tip**
>
> It might be helpful to imagine the `<<` operator (and `>>` operator) as a conveyor belt that moves data in the direction indicated. In this case, when the content is conveyed to `std::cout`, it gets output.

Here's another example where we print both text and the value of a variable in the same statement:

```
#include <iostream> // for std::cout

int main()
{
    int x{ 5 };
    std::cout << "x is equal to: " << x;
    return 0;
}
```

This program prints:

```
x is equal to: 5
```

> **Related content**

> We discuss what the *std::* prefix actually does in lesson 2.9 – Naming collisions and an introduction to namespaces.

Using `std::endl` to output a newline

What would you expect this program to print?

```
#include <iostream> // for std::cout

int main()
{
    std::cout << "Hi!";
    std::cout << "My name is Alex.";
    return 0;
}
```

You might be surprised at the result:

```
Hi!My name is Alex.
```

Separate output statements don't result in separate lines of output on the console.

If we want to print separate lines of output to the console, we need to tell the console to move the cursor to the next line. We can do that by outputting a newline. A **newline** is an OS-specific character or sequence of characters that moves the cursor to the start of the next line.

One way to output a newline is to output `std::endl` (which stands for "end line"):

```
#include <iostream> // for std::cout and std::endl

int main()
{
    std::cout << "Hi!" << std::endl; // std::endl will cause the cursor to move to the next line
    std::cout << "My name is Alex." << std::endl;

    return 0;
}
```

This prints:

```
Hi!
My name is Alex.
```

> **Tip**
>
> In the above program, the second `std::endl` isn't technically necessary, since the program ends immediately afterward. However, it serves a few useful purposes.
>
> First, it helps indicate that the line of output is a "complete thought" (as opposed to partial output that is completed somewhere later in the code). In this sense, it functions similarly to using a period in standard English.
>
> Second, it positions the cursor on the next line, so that if we later add additional lines of output (e.g. have the program say "bye!"), those lines will appear where we expect (rather than appended to the prior line of output).
>
> Third, after running an executable from the command line, some operating systems do not output a new line before showing the command prompt again. If our program does not end with the cursor on a new line, the command prompt may appear appended to the prior line of output, rather than at the start of a new line as the user would expect.

> **Best practice**
>
> Output a newline whenever a line of output is complete.

`std::cout` is buffered

Consider a rollercoaster ride at your favorite amusement park. Passengers show up (at some variable rate) and get in line. Periodically, a train arrives and boards passengers (up to the maximum capacity of the train). When the train is full, or when enough time has passed, the train departs with a batch of passengers, and the ride commences. Any passengers unable to board the current train wait for the next one.

This analogy is similar to how output sent to `std::cout` is typically processed in C++. Statements in our program request that output be sent to the console. However, that output is typically not sent to the console immediately. Instead, the requested output "gets in line", and is stored in a region of memory set aside to collect such requests (called a **buffer**). Periodically, the buffer is **flushed**, meaning all of the data collected in the buffer is transferred to its destination (in this case, the console).

> **Author's note**
>
> To use another analogy, flushing a buffer is kind of like flushing a toilet. All of your collected "output" is transferred to … wherever it goes next. Eew.

This also means that if your program crashes, aborts, or is paused (e.g. for debugging purposes) before the buffer is flushed, any output still waiting in the buffer will not be displayed.

> **Key insight**
>
> The opposite of buffered output is unbuffered output. With unbuffered output, each individual output request is sent directly to the output device.
>
> Writing data to a buffer is typically fast, whereas transferring a batch of data to an output device is comparatively slow. Buffering can significantly increase performance by batching multiple output requests together to minimize the number of times output has to be sent to the output device.

`std::endl` vs `\n`

Using `std::endl` is often inefficient, as it actually does two jobs: it outputs a newline (moving the cursor to the next line of the console), and it flushes the buffer (which is slow). If we output multiple lines of text ending with `std::endl`, we will get multiple flushes, which is slow and probably unnecessary.

When outputting text to the console, we typically don't need to explicitly flush the buffer ourselves. C++'s output system is designed to self-flush periodically, and it's both simpler and more efficient to let it flush itself.

To output a newline without flushing the output buffer, we use `\n` (inside either single or double quotes), which is a special symbol that the compiler interprets as a newline character. `\n` moves the cursor to the next line of the console without causing a flush, so it will typically perform better. `\n` is also more concise to type and can be embedded into existing double-quoted text.

Here's an example that uses `\n` in a few different ways:

```cpp
#include <iostream> // for std::cout

int main()
{
    int x{ 5 };
    std::cout << "x is equal to: " << x << '\n'; // single quoted (by itself) (conventional)
    std::cout << "Yep." << "\n";                 // double quoted (by itself) (unconventional but okay)
    std::cout << "And that's all, folks!\n";     // between double quotes in existing text (conventional)
    return 0;
}
```

This prints:

```
x is equal to: 5
Yep.
And that's all, folks!
```

When `\n` is not being embedded into an existing line of double-quoted text (e.g. `"hello\n"`), it is conventionally single quoted (`'\n'`).

> **For advanced readers**
>
> In C++, we use single quotes to represent single characters (such as `'a'` or `'$'`), and double-quotes to represent text (zero or more characters).
>
> Even though `'\n'` is represented in source code as two symbols, it is treated by the compiler as a single linefeed (LF) character (with ASCII value 10), and thus is conventionally single quoted (unless embedded into existing double-quoted text). We discuss this more in lesson 4.11 – Chars.
>
> When `'\n'` is output, the library doing the outputting is responsible for translating this single LF character into the appropriate newline sequence for the given OS. See Wikipedia for more information on OS conventions for newlines.

> **Author's note**
>
> Although unconventional, we believe it's fine to use (or even prefer) double quoted `"\n"` in standard output statements.
>
> This has two primary benefits:
>
> 1. It's simpler to double-quote all outputted text rather than having to determine what should be single-quoted and double-quoted.
> 2. More importantly, it helps avoid inadvertent multicharacter literals. We cover multicharacter literals and some of the unexpected output they can cause in lesson 4.11 – Chars.
>
> Single quotes should be preferred in non-output cases.

We'll cover what `'\n'` is in more detail when we get to the lesson on chars (4.11 – Chars).

> **Best practice**
>
> Prefer `\n` over `std::endl` when outputting text to the console.

> **Warning**
>
> `'\n'` uses a backslash (as do all special characters in C++), not a forward slash.
>
> Using a forward slash (e.g. `'/n'`) or including other characters inside the single quotes (e.g. `' \n'` or `'.\n'`) will result in unexpected behavior. For example, `std::cout << '/n';` will often print as `12142`, which probably isn't what you were expecting.

std::cin

`std::cin` is another predefined variable in the `iostream` library. Whereas `std::cout` prints data to the console (using the insertion operator `<<` to provide the data), `std::cin` (which stands for "character input") reads input from keyboard. We typically use the extraction operator `>>` to put the input data in a variable (which can then be used in subsequent statements).

```
#include <iostream>  // for std::cout and std::cin

int main()
{
    std::cout << "Enter a number: "; // ask user for a number

    int x{};       // define variable x to hold user input (and value-initialize it)
    std::cin >> x; // get number from keyboard and store it in variable x

    std::cout << "You entered " << x << '\n';
    return 0;
}
```

Try compiling this program and running it for yourself. When you run the program, line 5 will print "Enter a number:". When the code gets to line 8, your program will wait for you to enter input. Once you enter a number (and press enter), the number you enter will be assigned to variable `x`. Finally, on line 10, the program will print "You entered" followed by the number you just entered.

For example (entering the value 4 as input):

```
Enter a number: 4
You entered 4
```

This is an easy way to get keyboard input from the user, and we will use it in many of our examples going forward.

> **Tip**
>
> Note that you don't need to output `'\n'` when accepting a line of input, as the user will need to press the *enter* key to have their input accepted, and this will move the cursor to the next line of the console.

- If your screen closes immediately after entering a number, please see lesson 0.8 – A few common C++ problems for a solution.
- If you are using CLion and "You entered" has a space before it, this is a bug in CLion. See the CLion bug tracker for workarounds.

Just like it is possible to output more than one bit of text in a single line, it is also possible to input more than one value on a single line:

```
#include <iostream>  // for std::cout and std::cin

int main()
{
    std::cout << "Enter two numbers separated by a space: ";

    int x{}; // define variable x to hold user input (and value-initialize it)
    int y{}; // define variable y to hold user input (and value-initialize it)
    std::cin >> x >> y; // get two numbers and store in variable x and y respectively

    std::cout << "You entered " << x << " and " << y << '\n';

    return 0;
}
```

This produces the output:

```
Enter two numbers separated by a space: 5 6
You entered 5 and 6
```

Values entered should be separated by whitespace (spaces, tabs, or newlines).

> **Key insight**
>
> There's some debate over whether it's necessary to initialize a variable immediately before you give it a user provided value via another source (e.g. `std::cin`), since the user-provided value will just overwrite the initialization value. In line with our previous recommendation that variables should always be initialized, best practice is to initialize the variable first.

> **For advanced readers**
>
> The C++ I/O library does not provide a way to accept keyboard input without the user having to press *enter*. If this is something you desire, you'll have to use a third party library. For console applications, we'd recommend pdcurses, FXTUI, cpp-terminal, or notcurses. Many graphical user interface libraries have their own functions to do this kind of thing.

`std::cin` is buffered

In a prior section, we noted that outputting data is actually a two stage process:

- The data from each output request is added (to the end) of an output buffer.
- Later, data from (the front of) the output buffer is flushed to the output device (the console).

> **Key insight**
>
> Adding data to the end of a buffer and removing it from the front of a buffer ensures data is processed in the same order in which it was added. This is sometimes called FIFO (first in, first out).

Similarly, inputting data is also a two stage process:

- The individual characters you enter as input are added to the end of an input buffer (inside `std::cin`). The enter key (pressed to submit the data) is also stored as a `'\n'` character.
- The extraction operator `>>` removes characters from the front of the input buffer and converts them into a value that is assigned (via copy-assignment) to the associated variable. This variable can then be used in subsequent statements.

> **Key insight**
>
> Each line of input data in the input buffer is terminated by a `'\n'` character.

We'll demonstrate this using the following program:

```
#include <iostream>  // for std::cout and std::cin

int main()
{
    std::cout << "Enter two numbers: ";

    int x{};
    std::cin >> x;

    int y{};
    std::cin >> y;

    std::cout << "You entered " << x << " and " << y << '\n';

    return 0;
}
```

This program inputs to two variables (this time as separate statements). We'll run this program twice.

Run #1: When `std::cin >> x;` is encountered, the program will wait for input. Enter the

Introduction to Literals and Operators

Literals

Consider the following two statements:

```
std::cout << "Hello world!";
int x { 5 };
```

What are `"Hello world!"` and `5`? They are literals. A **literal** (also known as a **literal constant**) is a fixed value that has been inserted directly into the source code.

Literals and variables both have a value (and a type). Unlike a variable (whose value can be set and changed through initialization and assignment respectively), the value of a literal is fixed and cannot be changed. The literal `5` always has value `5`. This is why literals are called constants.

To further highlight the difference between literals and variables, let's examine this short program:

```
#include <iostream>

int main()
{
    std::cout << 5 << '\n'; // print the value of a literal

    int x { 5 };
    std::cout << x << '\n'; // print the value of a variable
    return 0;
}
```

On line 5, we're printing the value `5` to the console. When the compiler compiles this, it will generate code that causes `std::cout` to print the value `5`. This value `5` is compiled into the executable and can be used directly.

On line 7, we're creating a variable named `x`, and initializing it with value `5`. The compiler will generate code that copies the literal value `5` into whatever memory location is given to `x`. On line 8, when we print `x`, the compiler will generate code that causes `std::cout` to print the value at the memory location of `x` (which has value `5`).

Thus, both output statements do the same thing (print the value 5). But in the case of the literal, the value `5` can be printed directly. In the case of the variable, the value `5` must be fetched from the memory the variable represents.

This also explains why a literal is constant while a variable can be changed. A literal's value is placed directly in the executable, and the executable itself can't be changed after it is created. A variable's value is placed in memory, and the value of memory can be changed while the executable is running.

> **Key insight**
>
> Literals are values that are inserted directly into the source code. These values usually appear directly in the executable code (unless they are optimized out).
>
> Objects and variables represent memory locations that hold values. These values can be fetched on demand.

> **Related content**
>
> We talk more about literals in lesson 5.2 – Literals.

Operators

In mathematics, an **operation** is a process involving zero or more input values (called **operands**) that produces a new value (called an *output value*). The specific operation to be performed is denoted by a symbol called an **operator**.

For example, as children we all learn that *2 + 3* equals *5*. In this case, the literals *2* and *3* are the operands, and the symbol *+* is the operator that tells us to apply mathematical addition on the operands to produce the new value *5*.

In C++, operations work as you'd expect. For example:

```
#include <iostream>

int main()
{
    std::cout << 1 + 2 << '\n';

    return 0;
}
```

In this program, the literals `1` and `2` are operands to the plus (`+`) operator, which produces the output value `3`. This output value is then printed to the console. In C++, the output value of an operation is often called a **return value**.

You are likely already quite familiar with standard arithmetic operators from common usage in mathematics, including addition (`+`), subtraction (`-`), multiplication (`*`), and division (`/`). In C++, assignment (`=`) is an operator as well, as are insertion (`<<`), extraction (`>>`), and equality (`==`). While most operators have symbols for names (e.g. `+`, or `==`), there are also a number of operators that are keywords (e.g. `new`, `delete`, and `throw`).

> **Author's note**
>
> For reasons that will become clear when we discuss operators in more detail, for operators that are symbols, it is common to append the operator's symbol to the word *operator*.
>
> For example, the plus operator would be written `operator+`, and the extraction operator would be written `operator>>`.

The number of operands that an operator takes as input is called the operator's **arity**. Few people know what this word means, so don't drop it in a conversation and expect anybody to have any idea what you're talking about. Operators in C++ come in four different arities:

Unary operators act on one operand. An example of a unary operator is the `-` operator. For example, given `-5`, `operator-` takes literal operand `5` and flips its sign to produce new output value `-5`.

Binary operators act on two operands (often called *left* and *right*, as the left operand appears on the left side of the operator, and the right operand appears on the right side of the operator). An example of a binary operator is the `+` operator. For example, given `3 + 4`, `operator+` takes the left operand `3` and the right operand `4` and applies mathematical addition to produce new output value `7`. The insertion (`<<`) and extraction (`>>`) operators are binary operators, taking `std::cout` or `std::cin` on the left side, and the value to output or variable to input to on the right side.

Ternary operators act on three operands. There is only one of these in C++ (the conditional operator), which we'll cover later.

Nullary operators act on zero operands. There is also only one of these in C++ (the throw operator), which we'll also cover later.

Note that some operators have more than one meaning depending on how they are used. For example, `operator-` has two contexts. It can be used in unary form to invert a number's sign (e.g. to convert `5` to `-5`, or vice versa), or it can be used in binary form to do subtraction (e.g. `4 - 3`).

Chaining operators

Operators can be chained together such that the output of one operator can be used as the input for another operator. For example, given the following: `2 * 3 + 4`, the multiplication operator goes first, and converts left operand `2` and right operand `3` into return value `6` (which becomes the left operand for the plus operator). Next, the plus operator executes, and converts left operand `6` and right operand `4` into new value `10`.

We'll talk more about the order in which operators execute when we do a deep dive into the topic of operators. For now, it's enough to

know that the arithmetic operators execute in the same order as they do in standard mathematics: Parenthesis first, then Exponents, then Multiplication & Division, then Addition & Subtraction. This ordering is sometimes abbreviated *PEMDAS*, or expanded to the mnemonic "Please Excuse My Dear Aunt Sally".

> **Author's note**
>
> In some countries, PEMDAS is taught as PEDMAS, BEDMAS, BODMAS, or BIDMAS instead.

Return values and side effects

Most operators in C++ just use their operands to calculate a return value. For example, `-5` produces return value `-5` and `2 + 3` produces return value `5`. There are a few operators that do not produce return values (such as `delete` and `throw`). We'll cover what these do later.

Some operators have additional behaviors. An operator (or function) that has some observable effect beyond producing a return value is said to have a **side effect**. For example, `x = 5` has the side effect of assigning value `5` to variable `x`. The changed value of `x` is observable (e.g. by printing the value of `x`) even after the operator has finished executing. `std::cout << 5` has the side effect of printing `5` to the console. We can observe the fact that `5` has been printed to the console even after `std::cout << 5` has finished executing.

> **Nomenclature**
>
> In common language, the term "side effect" is typically used to mean a secondary (often negative or unexpected) result of some other thing happening (such as taking medicine). For example, a common side effect of taking oral antibiotics is diarrhea. As such, we often think of side effects as things we want to avoid, or things that are incidental to the primary goal.
>
> In C++, the term "side effect" has a different meaning: it is an observable effect of an operator or function beyond producing a return value.
>
> Since assignment has the observable effect of changing the value of an object, this is considered a side effect. We use certain operators (e.g. the assignment operator) primarily for their side effects (rather than the return value those operators produce). In such cases, the side effect is both beneficial and predictable (and it is the return value that is often incidental).

> **For advanced readers**
>
> For the operators that we call primarily for their return values (e.g. `operator+` or `operator*`), it's usually obvious what their return values will be (e.g. the sum or product of the operands).
>
> For the operators we call primarily for their side effects (e.g. `operator=` or `operator<<`), it's not always obvious what return values they produce (if any). For example, what return value would you expect `x = 5` to have?
>
> Both `operator=` and `operator<<` (when used to output values to the console) return their left operand. Thus, `x = 5` returns `x`, and `std::cout << 5` returns `std::cout`. This is done so that these operators can be chained.
>
> For example, `x = y = 5` evaluates as `x = (y = 5)`. First `y = 5` assigns `5` to `y`. This operation then returns `y`, which can then be assigned to `x`.
>
> `std::cout << "Hello " << "world!"` evaluates as `(std::cout << "Hello ") << "world!"`. This first prints `"Hello "` to the console. This operation returns `std::cout`, which can then be used to print `"world!"` to the console as well.
>
> We talk more about the order in which operators evaluate in lesson 6.1 – Operator precedence and associativity.

Introduction to Objects and Variables

Data and values

In lesson 1.1 – Statements and the structure of a program, you learned that the majority of instructions in a program are statements, and that functions are groups of statements that execute sequentially. The statements inside the function perform actions that (hopefully) generate whatever result the program was designed to produce.

But how do programs actually produce results? They do so by manipulating (reading, changing, and writing) data. In computing, **data** is any information that can be moved, processed, or stored by a computer.

> **Key insight**
>
> Programs are collections of instructions that manipulate data to produce a desired result.

Computer programs (both in source code format and compiled) are technically data too, since they can be moved, processed, and stored. However, in the context of a computer program, we typically use the term "code" to mean the program itself, and "data" to mean the information that the program works with to produce a result.

A program can acquire data to work with in many ways: from a file or database, over a network, from the user providing input on a keyboard, or from the programmer putting data directly into the source code of the program itself. In the "Hello world" program from the aforementioned lesson, the text "Hello world!" was inserted directly into the source code of the program, providing data for the program to use. The program then manipulates this data by sending it to the monitor to be displayed.

In programming, a single piece of data is called a **value** (sometimes called a **data value**). Common examples of values include:

- Numbers (e.g. `5` or `-6.7`).
- Characters, which are placed between single-quotes (e.g. `'H'` or `'$'`). Only a single symbol may be used.
- Text, which must be placed between double-quotes (e.g. `"Hello"` or `"H"`). Text can contain 0 or more characters.

> **Key insight**
>
> Values placed in single-quotes are interpreted by the compiler as character values. Values placed in double-quotes are interpreted by the compiler as text values. Numeric values are not quoted.

Values that are placed directly into the source code are called **literals**. We cover literals in detail in lesson 5.2 – Literals.

One of the most common thing to do with values is print them to the screen. For example:

```
#include <iostream> // for std::cout

int main()
{
    std::cout << 5;       // print the literal number `5`
    std::cout << -6.7;    // print the literal number `-6.7`
    std::cout << 'H';     // print the literal character `H`
    std::cout << "Hello"; // print the literal text `Hello`

    return 0;
}
```

If a character value or text value is not properly quoted, the compiler will try to interpret that value as if it were C++ code. That will almost always result in a compilation error.

Literals are the easiest way to provide values for your program, but they have some shortcomings. Literals are read-only values, so their values can't be modified. Thus, if we want to store data in memory, we need some other way to do this.

Random Access Memory

The main memory in a computer is called **Random Access Memory** (often called **RAM** for short). When we run a program, the operating system loads the program into RAM. Any data that is hardcoded into the program itself (e.g. text such as "Hello, world!") is loaded at this point.

The operating system also reserves some additional RAM for the program to use while it is running. Common uses for this memory are to store values entered by the user, to store data read in from a file or network, or to store values calculated while the program is running (e.g. the sum of two values) so they can be used again later.

You can think of RAM as a series of numbered boxes that can be used to store data while the program is running.

In some older programming languages (like Applesoft BASIC), you could directly access these boxes (e.g. you could write a statement to "go get the value stored in membox number 7532").

Objects and variables

In C++, direct memory access is discouraged. Instead, we access memory indirectly through an object. An **object** represents a region of storage (typically RAM or a CPU register) that can hold a value. Objects also have associated properties (that we'll cover in future lessons).

How the compiler and operating system work to assign memory to objects is beyond the scope of this lesson. But the key point here is that rather than say "go get the value stored in mailbox number 7532", we can say, "go get the value stored by this object" and let the compiler figure out where and how to retrieve the value. This means we can focus on using objects to store and retrieve values, and not have to worry about where in memory those objects are actually being placed.

Although objects in C++ can be unnamed (anonymous), more often we name our objects using an identifier. An object with a name is called a **variable**.

> **Key insight**
>
> An object is used to store a value in memory. A variable is an object that has a name (identifier).

Naming our objects let us refer to those objects again later in the program.

> **Nomenclature**
>
> In general programming, the term *object* typically refers to an unnamed object in memory, a variable, or a function. In C++, the term *object* has a narrower definition that excludes functions. When we use the term object in this tutorial series, this narrower C++ definition is the one we mean.

Variable definition

In order to use a variable in our program, we need to tell the compiler that we want one. The most common way to do this is by use of a special kind of declaration statement called a **definition** (we'll clarify the difference between a declaration and definition later, in lesson 2.7 – Forward declarations and definitions).

> **Key insight**
>
> A definition statement can be used to tell the compiler that we want to use a variable in our program.

Here's an example of defining a variable named `x`:

```
int x; // define a variable named x (of type int)
```

At **compile-time** (when the program is being compiled), when encountering this statement, the compiler makes a note to itself that we want a variable with the name `x`, and that the variable has the data type `int` (more on data types in a moment). From that point forward (with some limitations that we'll talk about in a future lesson), whenever we use the identifier `x` in our code, the compiler will know that we are

referring to this variable.

The compiler handles all of the other details about this variable for us, including determining how much memory the object will need, in what kind of storage the object will be placed (e.g. in RAM or a CPU register), where it will be placed relative to other objects, when it will be created and destroyed, etc...

A variable created via a definition statement is said to be **defined** at the point where the definition statement is placed. For now, your variables should be defined inside functions (such as within `main()`).

Here is a full program containing a variable definition statement:

```
int main()
{
    int x; // definition of variable x

    return 0;
}
```

Variable creation

At **runtime** (when the program is loaded into memory and run), each object is given an actual storage location (such as RAM, or a CPU register) that it can use to store values. The process of reserving storage for an object's use is called **allocation**. Once allocation has occurred, the object has been created and can be used.

For the sake of example, let's say that variable `x` is instantiated at memory location 140. Whenever the program uses variable `x`, it will access the value in memory location 140.

> **Key insight**
>
> An object is "created" once actual storage has been reserved for the object's use.

When the above program is run, execution starts at the top of `main()`. Memory for `x` is allocated. Then the program ends.

Data types

So far, we've covered that objects are regions of storage that can store a data value (how exactly data is stored is a topic for a future lesson). A **data type** (more commonly just called a **type**) determines what kind of value (e.g. a number, a letter, text, etc...) the object will store.

In the above example, our variable `x` was given type `int`, which means variable `x` will store an integer value. An **integer** is a number that can be written without a fractional component, such as `4`, `27`, `0`, `-2`, or `-12`. For short, we can say that `x` is an `integer variable`.

In C++, the type of an object must be known at compile-time, and that type can not be changed without recompiling the program. This means an integer variable can only hold integer values. If you want to store some other kind of value, you'll need to use a different type.

> **Key insight**
>
> The data type of an object must be known at compile-time (so the compiler knows how much memory that object requires).

Integers are just one of many types that C++ supports out of the box. For illustrative purposes, here's another example of defining a variable using data type `double`:

```
double width; // define a variable named width, of type double
```

C++ also allows you to create your own custom types. This is something we'll do a lot of in future lessons, and it's part of what makes C++ powerful.

For these introductory chapters, we'll stick with integer variables because they are conceptually simple, but we'll explore many of the other types C++ has to offer (including `double`) soon.

Defining multiple variables

It is possible to define multiple variables *of the same type* in a single statement by separating the names with a comma. The following code snippet:

```
int a;
int b;
```

is effectively the same as this one:

```
int a, b;
```

When defining multiple variables this way, there are three common mistakes that new programmers tend to make (neither serious, since the compiler will catch these and ask you to fix them):

The first mistake is giving each variable a type when defining variables in sequence.

```
int a, int b; // wrong (compiler error)

int a, b; // correct
```

The second mistake is to try to define variables of different types in the same statement, which is not allowed. Variables of different types must be defined in separate statements.

```
int a, double b; // wrong (compiler error)

int a; double b; // correct (but not recommended)

// correct and recommended (easier to read)
int a;
double b;
```

We'll discuss the third issue next lesson, when we cover initialization of variables.

> **Best practice**
>
> Although the language allows you to do so, avoid defining multiple variables of the same type in a single statement. Instead, define each variable in a separate statement on its own line (and then use a single-line comment to document what it is used for).

Summary

In C++, we use objects to access memory. A named object is called a variable. Each variable has an identifier, a type, and a value (and some other attributes that aren't relevant here). A variable's type is used to determine how the value in memory should be interpreted.

Variables are actually created at runtime, when memory is allocated for their use.

In the next lesson, we'll look at how to give values to our variables and how to actually use them.

Keywords and Naming Identifiers

Keywords

C++ reserves a set of 92 words (as of C++23) for its own use. These words are called **keywords** (or reserved words), and each of these keywords has a special meaning within the C++ language.

Here is a list of all the C++ keywords (through C++23):

- alignas
- alignof
- and
- and_eq
- asm
- auto
- bitand
- bitor
- bool
- break
- case
- catch
- char
- char8_t (since C++20)
- char16_t
- char32_t
- class
- compl
- concept (since C++20)
- const
- consteval (since C++20)
- constexpr
- constinit (since C++20)
- const_cast
- continue
- co_await (since C++20)
- co_return (since C++20)
- co_yield (since C++20)
- decltype
- default
- delete
- do
- double
- dynamic_cast
- else
- enum
- explicit
- export

- extern
- false
- float
- for
- friend
- goto
- if
- inline
- int
- long
- mutable
- namespace
- new
- noexcept
- not
- not_eq
- nullptr
- operator
- or
- or_eq
- private
- protected
- public
- register
- reinterpret_cast
- requires (since C++20)
- return
- short
- signed
- sizeof
- static
- static_assert
- static_cast
- struct
- switch
- template
- this
- thread_local
- throw
- true
- try
- typedef
- typeid
- typename
- union

- unsigned
- using
- virtual
- void
- volatile
- wchar_t
- while
- xor
- xor_eq

The keywords marked (C++20) were added in C++20. If your compiler is not C++20 compliant (or does have C++20 functionality, but it's turned off by default), these keywords may not be functional.

C++ also defines special identifiers: *override*, *final*, *import*, and *module*. These have a specific meaning when used in certain contexts but are not reserved otherwise.

You have already run across some of these keywords, including *int* and *return*. Along with a set of operators, these keywords and special identifiers define the entire language of C++ (preprocessor commands excluded). Because keywords and special identifiers have special meaning, your IDEs will likely change the text color of these words to make them stand out from other identifiers.

By the time you are done with this tutorial series, you will understand what almost all of these words do!

Identifier naming rules

As a reminder, the name of a variable (or function, type, or other kind of item) is called an identifier. C++ gives you a lot of flexibility to name identifiers as you wish. However, there are a few rules that must be followed when naming identifiers:

- The identifier can not be a keyword. Keywords are reserved.
- The identifier can only be composed of letters (lower or upper case), numbers, and the underscore character. That means the name can not contain symbols (except the underscore) nor whitespace (spaces or tabs).
- The identifier must begin with a letter (lower or upper case) or an underscore. It can not start with a number.
- C++ is case sensitive, and thus distinguishes between lower and upper case letters. `nvalue` is different than `nValue` is different than `NVALUE`.

Identifier naming best practices

Now that you know how you *can* name a variable, let's talk about how you *should* name a variable (or function).

1. It is conventional in C++ that variable names should begin with a lowercase letter. If the variable name is a single word or acronym, the whole thing should be written in lowercase letters.

```
int value; // conventional

int Value; // unconventional (should start with lower case letter)
int VALUE; // unconventional (should start with lower case letter and be in all lower case)
int VaLuE; // unconventional (see your psychiatrist) ;)
```

Most often, function names are also started with a lowercase letter (though there's some disagreement on this point). We'll follow this convention, since function *main* (which all programs must have) starts with a lowercase letter, as do all of the functions in the C++ standard library.

Identifier names that start with a capital letter are typically used for user-defined types (such as structs, classes, and enumerations, all of which we will cover later).

If the variable or function name is multi-word, there are two common conventions: words separated by underscores (sometimes called snake_case), or intercapped (sometimes called camelCase, since the capital letters stick up like the humps on a camel).

```
int my_variable_name;   // conventional (separated by underscores/snake_case)
int my_function_name(); // conventional (separated by underscores/snake_case)

int myVariableName;     // conventional (intercapped/camelCase)
int myFunctionName();   // conventional (intercapped/camelCase)

int my variable name;   // invalid (whitespace not allowed)
int my function name(); // invalid (whitespace not allowed)

int MyVariableName;     // unconventional (should start with lower case letter)
int MyFunctionName();   // unconventional (should start with lower case letter)
```

In this tutorial, we will typically use the intercapped approach because it's easier to read (it's easy to mistake an underscore for a space in dense blocks of code). But it's common to see either – the C++ standard library uses the underscore method for both variables and functions. Sometimes you'll see a mix of the two: underscores used for variables and intercaps used for functions.

It's worth noting that if you're working in someone else's code, it's generally considered better to match the style of the code you are working in than to rigidly follow the naming conventions laid out above.

> **Best practice**
>
> When working in an existing program, use the conventions of that program (even if they don't conform to modern best practices). Use modern best practices when you're writing new programs.

2. Avoid naming your identifiers starting with an underscore. Although syntactically legal, these names are typically reserved for OS, library, and/or compiler use.
3. The name of your identifiers should make clear what the value they are holding means (particularly if the units aren't obvious). Identifiers should be named in a way that would help someone who has no idea what your code does be able to figure it out as quickly as possible. In 3 months, when you look at your program again, you'll have forgotten how it works, and you'll thank yourself for picking variable names that make sense.

However, giving a trivial identifier an overly-complex name impedes overall understanding of what the program is doing almost as much as giving a non-trivial identifier an inadequate name. A good rule of thumb is to make the length of an identifier proportional to how specific and accessible the identifier is. This means:

- An identifier that exists for only a few statements (e.g. in the body of a short function) can have a shorter name.
- An identifier that is accessible from anywhere might benefit from a longer name.
- An identifier that represents a non-specific number (e.g. anything the user provides) can have a shorter name.
- An identifier that represents a specific value (e.g. the length of an inseam in millimeters) should have a longer name.

Identifier	Rating	Notes
int ccount	Bad	What does the c before “count” stand for?
int customerCount	Good	Clear what we’re counting
int i	Either	Okay if use is trivial, bad otherwise
int index	Either	Okay if obvious what we’re indexing
int totalScore	Either	Okay if there’s only one thing being scored, otherwise too ambiguous
int _count	Bad	Do not start names with underscore
int count	Either	Okay if obvious what we’re counting
int data	Bad	What kind of data?
int time	Bad	Is this in seconds, minutes, or hours?
int minutesElapsed	Either	Okay if obvious what this is elapsed from
int x1, x2	Either	Okay if use is trivial, bad otherwise
int userinput1, userinput2	Bad	Hard to differentiate between the two due to long name
int numApples	Good	Descriptive
int monstersKilled	Good	Descriptive

4. Avoid abbreviations, except when they are common and unambiguous (e.g. `num`, `cm`, `idx`).

> **Key insight**
>
> Code is read more often than it is written, so any time saved while writing the code is time that every reader, including future you, will waste while reading it. If you’re looking to write code faster, use your editor’s auto-complete feature.

5. For variable declarations, it can be useful to use a comment to describe what a variable is going to be used for, or to explain anything else that might not be obvious. For example, say we’ve declared a variable that is supposed to store the number of characters in a piece of text. Does the text “Hello World!” have 10, 11, 12 characters? It depends on whether we’re including whitespace or punctuation. Rather than naming the variable `numCharsIncludingWhitespaceAndPunctuation`, which is rather lengthy, a well placed comment on or above the declaration line should help the user figure it out:

```
// a count of the number of chars in a piece of text, including whitespace and punctuation
int numChars {};
```

Statements and the structure of a program

Chapter introduction

Welcome to the first primary chapter of these C++ tutorials!

In this chapter, we'll take a first look at a number of topics that are essential to every C++ program. Because there are quite a few topics to cover, we'll cover most at a fairly shallow level (just enough to get by). The goal of this chapter is to help you understand how basic C++ programs are constructed. By the end of the chapter, you will be able to write your own simple programs.

In future chapters, we'll revisit the majority of these topics and explore them in more detail. We'll also introduce new concepts that build on top of these.

In order to keep the lesson lengths manageable, topics may be split over several subsequent lessons. If you feel like some important concept isn't covered in a lesson, or you have a question that isn't answered in the current lesson, it's possible that it's covered in the next lesson.

Statements

A computer program is a sequence of instructions that tell the computer what to do. A **statement** is a type of instruction that causes the program to *perform some action*.

Statements are by far the most common type of instruction in a C++ program. This is because they are the smallest independent unit of computation in the C++ language. In that regard, they act much like sentences do in natural language. When we want to convey an idea to another person, we typically write or speak in sentences (not in random words or syllables). In C++, when we want to have our program do something, we typically write statements.

Most (but not all) statements in C++ end in a semicolon. If you see a line that ends in a semicolon, it's probably a statement.

In a high-level language such as C++, a single statement may compile into many machine language instructions.

> **For advanced readers**
>
> There are many different kinds of statements in C++:
>
> - Declaration statements
> - Jump statements
> - Expression statements
> - Compound statements
> - Selection statements (conditionals)
> - Iteration statements (loops)
> - Try blocks
>
> By the time you're through with this tutorial series, you'll understand what all of these are!

Functions and the `main` function

In C++, statements are typically grouped into units called functions. A **function** is a collection of statements that get executed sequentially (in order, from top to bottom). As you learn to write your own programs, you'll be able to create your own functions and mix and match statements in any way you please (we'll show how in a future lesson).

> **Rule**
>
> Every C++ program must have a special function named **main** (all lower case letters).

When the program is run, the statements inside of `main` are executed in sequential order.

Programs typically terminate (finish running) after the last statement inside function `main` has been executed (though programs may abort early in some circumstances, or do some cleanup afterwards).

Functions are typically written to do a specific job or perform some useful action. For example, a function named `max` might contain statements that figures out which of two numbers is larger. A function named `calculateGrade` might calculate a student's grade from a set of test scores. A function named `printEmployee` might print an employee's information to the console. We will talk a lot more about functions soon, as they are the most commonly used organizing tool in a program.

> **Nomenclature**
>
> When discussing functions, it's fairly common shorthand to append a pair of parenthesis to the end of the function's name. For example, if you see the term `main()` or `doSomething()`, this is shorthand for functions named `main` or `doSomething` respectively. This helps differentiate functions from other things with names (such as variables) without having to write the word "function" each time.

In programming, the name of a function (or object, type, template, etc…) is called its **identifier**.

Characters and text

The earliest computers were designed primarily for mathematical calculations and data processing. As hardware improved, networking became accessible, and consumer software evolved, computers also became valuable tools for written communication.

In written language, the most basic unit of communication is the character. To simplify slightly, a **character** is a written symbol or mark, such as a letter, digit, punctuation mark, or mathematical symbol. When we tap an alphabetic or numeric key on our keyboard, a character is produced as a result, which can then be displayed on the screen. The following are all characters: `a`, `2`, `$`, and `=`.

In many cases, such as when writing words or sentences, we want to make use of more than one character. A sequence of characters is called **text** (also called a **string** in programming contexts).

> **Nomenclature**
>
> Conventionally, the term "text" is also used to mean **plain text**, which is text that contains only characters that appear on a standard keyboard, with no special formatting or styling. For example, plain text cannot represent bold words, as that requires styling.
>
> Our C++ programs are written as plain text.

> **For advanced readers**
>
> Computers have an additional type of character, called a "control character". These are characters that have special meaning to the computer system, but either aren't intended to be displayed, or display as something other than a single visible symbol. Examples of well-known control characters include "escape" (which doesn't display anything), "tab" (which displays as some number of spaces), and "backspace" (which erases the previous character).

> **Related content**
>
> We discuss the outputting of characters and text in lesson 1.5 – Introduction to iostream: cout, cin, and endl. We discuss characters (including control characters) in more detail in lesson 4.11 – Chars.

Dissecting Hello world!

Now that you have a brief understanding of what statements and functions are, let's return to our "Hello world" program and take a high-level look at what each line does in more detail.

```
#include <iostream>

int main()
{
   std::cout << "Hello world!";
   return 0;
}
```

Line 1 is a special type of line called a preprocessor directive. This `#include` preprocessor directive indicates that we would like to use the contents of the `iostream` library, which is the part of the C++ standard library that allows us to read and write text from/to the console. We need this line in order to use `std::cout` on line 5. Excluding this line would result in a compile error on line 5, as the compiler wouldn't otherwise know what `std::cout` is.

Line 2 is blank, and is ignored by the compiler. This line exists only to help make the program more readable to humans (by separating the `#include` preprocessor directive and the subsequent parts of the program).

Line 3 tells the compiler that we're going to write (define) a function whose name (identifier) is `main`. As you learned above, every C++ program must have a `main` function or it will fail to link. This function will produce a value whose type is `int` (an integer).

Lines 4 and 7 tell the compiler which lines are part of the *main* function. Everything between the opening curly brace on line 4 and the closing curly brace on line 7 is considered part of the `main` function. This is called the function body.

Line 5 is the first statement within function `main`, and is the first statement that will execute when we run our program. `std::cout` (which stands for "character output") and the `<<` operator allow us to display information on the console. In this case, we're displaying the text `Hello world!`. This statement creates the visible output of the program.

Line 6 is a return statement. When an executable program finishes running, the program sends a value back to the operating system in order to indicate whether it ran successfully or not. This particular return statement returns the integer value `0` to the operating system, which means "everything went okay!". This is the last statement in the program that executes.

All of the programs we write will follow this general template, or a variation on it.

> **Author's note**
>
> If parts (or all) of the above explanation are confusing, that's to be expected at this point. This was just to provide a quick overview. Subsequent lessons will dig into all of the above topics, with plenty of additional explanation and examples.

You can compile and run this program yourself, and you will see that it outputs the following to the console:

```
Hello world!
```

If you run into issues compiling or executing this program, check out lesson 0.8 – A few common C++ problems.

Syntax and syntax errors

In the English language, sentences are constructed according to specific grammatical rules that you probably learned in English class in school. For example, in writing, normal sentences end in a period. The set of rules that describe how specific words (and punctuation) can be arranged to form valid sentences in a language is called **syntax**. For example, "My house painted is blue" is a syntax error, because the ordering of the words is unconventional. "All your base are belong to us!" is another notable one.

The C++ programming language also has a syntax, which describes how the elements of your program must be written and arranged in order for the program to be considered valid. When you compile your program, the compiler is responsible for making sure your program follows these syntactical rules. If your program does something that deviates from the syntax of the language, the compiler will halt compilation and issue a *syntax error*.

Unlike the English language, which allows for a lot of ambiguity, the syntax rules of C++ are strictly defined and upheld. Syntax errors are common. Fortunately, such errors are typically straightforward to find and fix, as the compiler will generally point you right at them. Compilation of a program will only complete once all syntax errors are resolved.

Since the syntax for most statements requires those statements to end in a semicolon, let's see what happens if we omit the semicolon on line 5 of the "Hello world" program, like this:

```
#include <iostream>

int main()
{
   std::cout << "Hello world!"
   return 0;
}
```

Feel free to compile this ill-formed program yourself.

When compiled using Clang, the following error message is generated:

```
prog.cc:5:31: error: expected ';' after expression
```

Clang is telling us that on line 5 at the 31st character, the syntax rules require a semicolon, but we did not provide one. Therefore, compilation was halted with an error.

When compiled with Visual Studio instead, the compiler produces this compilation error:

```
c:\vcprojects\hello.cpp(6): error C2143: syntax error : missing ';' before 'return'
```

Note that Visual Studio says the error was encountered on line 6 (instead of on line 5). So who is right? Both are, in a way.

Clang knows we conventionally put semicolons at the end of statements, so it is reporting that the error is on line 5 based on the assumption that we will do so. Visual Studio is opting to report the line it was compiling when it determined that the error occurred (on line 6, when it encountered `return` instead of the expected semicolon).

> **Key insight**
>
> Compilers will sometimes report that an error has occurred on the line after the one where we'd conventionally fix the issue. If you can't find the error on the line indicated, check the prior line.

To see other different error messages, experiment with deleting characters or even whole lines from the "Hello world" program. Also try restoring the missing semicolon at the end of line 5, and then deleting lines 1, 3, or 4 to see what happens.

Uninitialized variables

Unlike some programming languages, C/C++ does not automatically initialize most variables to a given value (such as zero). When a variable that is not initialized is given a memory address to use to store data, the default value of that variable is whatever (garbage) value happens to already be in that memory address! A variable that has not been given a known value (through initialization or assignment) is called an **uninitialized variable**.

> **Nomenclature**
>
> Many readers expect the terms "initialized" and "uninitialized" to be strict opposites, but they aren't quite! In common language, "initialized" means the object was provided with an initial value at the point of definition. "Uninitialized" means the object has not been given a known value yet (through any means, including assignment). Therefore, an object that is not initialized but is then assigned a value is no longer *uninitialized* (because it has been given a known value).
>
> To recap:
>
> - Initialized = The object is given a known value at the point of definition.
> - Assignment = The object is given a known value beyond the point of definition.
> - Uninitialized = The object has not been given a known value yet.
>
> Relatedly, consider this variable definition:
>
> ```
> int x;
> ```
>
> In lesson 1.4 – Variable assignment and initialization, we noted that when no initializer is provided, the variable is default-initialized. In most cases (such as this one), default-initialization performs no actual initialization. Thus we'd say `x` is uninitialized. We're focused on the outcome (the object has not been given a known value), not the process.

> **As an aside...**
>
> This lack of initialization is a performance optimization inherited from C, back when computers were slow. Imagine a case where you were going to read in 100,000 values from a file. In such case, you might create 100,000 variables, then fill them with data from the file.
>
> If C++ initialized all of those variables with default values upon creation, this would result in 100,000 initializations (which would be slow), and for little benefit (since you're overwriting those values anyway).
>
> For now, you should always initialize your variables because the cost of doing so is minuscule compared to the benefit. Once you are more comfortable with the language, there may be certain cases where you omit the initialization for optimization purposes. But this should always be done selectively and intentionally.

Using the values of uninitialized variables can lead to unexpected results. Consider the following short program:

```
#include <iostream>

int main()
{
    // define an integer variable named x
    int x; // this variable is uninitialized because we haven't given it a value

    // print the value of x to the screen
    std::cout << x << '\n'; // who knows what we'll get, because x is uninitialized

    return 0;
}
```

In this case, the computer will assign some unused memory to *x*. It will then send the value residing in that memory location to *std::cout*, which will print the value (interpreted as an integer). But what value will it print? The answer is "who knows!", and the answer may (or may not) change every time you run the program. When the author ran this program in Visual Studio, *std::cout* printed the value `7177728` one time, and `5277592` the next. Feel free to compile and run the program yourself (your computer won't explode).

> **Warning**
>
> Some compilers, such as Visual Studio, *will* initialize the contents of memory to some preset value when you're using a debug build configuration. This will not happen when using a release build configuration. Therefore, if you want to run the above program yourself, make sure you're using a *release build configuration* (see lesson 0.9 – Configuring your compiler: Build configurations for a reminder on how to do that).
>
> For example, if you run the above program in a Visual Studio debug configuration, it will consistently print -858993460, because that's the value (interpreted as an integer) that Visual Studio initializes memory with in debug configurations.

Most modern compilers will attempt to detect if a variable is being used without being given a value. If they are able to detect this, they will generally issue a compile-time warning or error. For example, compiling the above program on Visual Studio produced the following warning:

```
c:\VCprojects\test\test.cpp(11) : warning C4700: uninitialized local variable 'x' used
```

If your compiler won't let you compile and run the above program (e.g. because it treats the issue as an error), here is a possible solution to get around this issue:

```
#include <iostream>

void doNothing(int&) // Don't worry about what & is for now, we're just using it to trick the compiler into thinking variable x is used
{
}

int main()
{
    // define an integer variable named x
    int x; // this variable is uninitialized

    doNothing(x); // make the compiler think we're assigning a value to this variable

    // print the value of x to the screen (who knows what we'll get, because x is uninitialized)
    std::cout << x << '\n';

    return 0;
}
```

Using uninitialized variables is one of the most common mistakes that novice programmers make, and unfortunately, it can also be one of the most challenging to debug (because the program may run fine anyway if the uninitialized variable happened to get assigned to a spot of memory that had a reasonable value in it, like 0).

This is the primary reason for the "always initialize your variables" best practice.

Undefined behavior

Using the value from an uninitialized variable is our first example of undefined behavior. **Undefined behavior** (often abbreviated **UB**) is the result of executing code whose behavior is not well-defined by the C++ language. In this case, the C++ language doesn't have any rules determining what happens if you use the value of a variable that has not been given a known value. Consequently, if you actually do this, undefined behavior will result.

Code implementing undefined behavior may exhibit *any* of the following symptoms:

- Your program produces different results every time it is run.
- Your program consistently produces the same incorrect result.
- Your program behaves inconsistently (sometimes produces the correct result, sometimes not).
- Your program seems like it's working but produces incorrect results later in the program.
- Your program crashes, either immediately or later.
- Your program works on some compilers but not others.
- Your program works until you change some other seemingly unrelated code.

Or, your code may actually produce the correct behavior anyway.

> **Author's note**
>
> Undefined behavior is like a box of chocolates. You never know what you're going to get!

C++ contains many cases that can result in undefined behavior if you're not careful. We'll point these out in future lessons whenever we encounter them. Take note of where these cases are and make sure you avoid them.

> **Rule**
>
> Take care to avoid all situations that result in undefined behavior, such as using uninitialized variables.

> **Author's note**
>
> One of the most common types of comment we get from readers says, "You said I couldn't do X, but I did it anyway and my program works! Why?".
>
> There are two common answers. The most common answer is that your program is actually exhibiting undefined behavior, but that undefined behavior just happens to be producing the result you wanted anyway… for now. Tomorrow (or on another compiler or machine) it might not.
>
> Alternatively, sometimes compiler authors take liberties with the language requirements when those requirements may be more restrictive than needed. For example, the standard may say, "you must do X before Y", but a compiler author may feel that's unnecessary, and make Y work even if you don't do X first. This shouldn't affect the operation of correctly written programs, but may cause incorrectly written programs to work anyway. So an alternate answer to the above question is that your compiler may simply be not following the standard! It happens. You can avoid much of this by making sure you've turned compiler extensions off, as described in lesson 0.10 – Configuring your compiler: Compiler extensions.

Implementation-defined behavior and unspecified behavior

A specific compiler and the associated standard library it comes with are called an **implementation** (as these are what actually implements the C++ language). In some cases, the C++ language standard allows the implementation to determine how some aspect of the language will behave, so that the compiler can choose a behavior that is efficient for a given platform. Behavior that is defined by the implementation is called **implementation-defined behavior**. Implementation-defined behavior must be documented and consistent for a given implementation.

Let's look at a simple example of implementation-defined behavior:

```
#include <iostream>

int main()
{
    std::cout << sizeof(int) << '\n'; // print how many bytes of memory an int value takes

    return 0;
}
```

On most platforms, this will produce `4`, but on others it may produce `2`.

> **Related content**
>
> We discuss `sizeof()` in lesson 4.3 – Object sizes and the sizeof operator.

Unspecified behavior is almost identical to implementation-defined behavior in that the behavior is left up to the implementation to define, but the implementation is not required to document the behavior.

We generally want to avoid implementation-defined and unspecified behavior, as it means our program may not work as expected if compiled on a different compiler (or even on the same compiler if we change project settings that affect how the implementation behaves!)

> **Best practice**
>
> Avoid implementation-defined and unspecified behavior whenever possible, as they may cause your program to malfunction on other implementations.

> **Related content**
>
> We show examples of unspecified behavior in lesson 6.1 – Operator precedence and associativity.

Variable Assignment and Initialization

In the previous lesson (1.3 – Introduction to objects and variables), we covered how to define a variable that we can use to store values. In this lesson, we'll explore how to actually put values into variables.

As a reminder, here's a short program that first allocates a single integer variable named `x`, then allocates two more integer variables named `y` and `z`:

```
int main()
{
    int x;    // define an integer variable named x (preferred)
    int y, z; // define two integer variables, named y and z

    return 0;
}
```

As a reminder, it is preferred to define one variable per line. We'll come back to cases where we define multiple variables later in this lesson.

Variable assignment

After a variable has been defined, you can give it a value (in a separate statement) using the `=` operator. This process is called **assignment**, and the `=` operator is called the **assignment operator**.

```
int width; // define an integer variable named width
width = 5; // assignment of value 5 into variable width

// variable width now has value 5
```

By default, assignment copies the value on the right-hand side of the `=` operator to the variable on the left-hand side of the operator. This is called **copy-assignment**.

Once a variable has been given a value, the value of that variable can be printed via `std::cout` and the `<<` operator.

Assignment can be used whenever we want to change the value held by a variable. Here's an example where we use assignment twice:

```
#include <iostream>

int main()
{
    int width; // define a variable named width
    width = 5; // copy assignment of value 5 into variable width

    std::cout << width; // prints 5

    width = 7; // change value stored in variable width to 7

    std::cout << width; // prints 7

    return 0;
}
```

This prints:

```
57
```

When this program is run, execution begins at the top of the `main` function and proceeds sequentially. First, memory for variable `width` is allocated. We then assign `width` the value `5`. When we output the value of `width`, it prints `5` to the console. When we then assign value `7` to `width`, any prior value (in this case `5`) is overwritten. Thus when we output `width` again, this time it prints `7`.

Normal variables can only hold one value at a time.

> **Warning**
>
> One of the most common mistakes that new programmers make is to confuse the assignment operator (`=`) with the equality operator (`==`). Assignment (`=`) is used to assign a value to a variable. Equality (`==`) is used to test whether two operands are equal in value.

Variable initialization

One downside of assignment is that assigning a value to a just-defined object requires two statements: one to define the variable, and another to assign the value.

These two steps can be combined. When an object is defined, you can optionally provide an initial value for the object. The process of specifying an initial value for an object is called **initialization**, and the syntax used to initialize an object is called an **initializer**. Informally, the initial value is often called an "initializer" as well.

For example, the following statement both defines a variable named `width` (of type `int`) and initializes it with the value `5`:

```
#include <iostream>

int main()
{
    int width { 5 };    // define variable width and initialize with initial value 5
    std::cout << width; // prints 5

    return 0;
}
```

In the above initialization of variable `width`, `{ 5 }` is the initializer, and `5` is the initial value.

> **Key insight**
>
> Initialization provides an initial value for a variable. Think "initial-ization".

Different forms of initialization

Unlike assignment (which is generally straightforward), initialization in C++ is surprisingly complex. So we'll present a simplified view here to get started.

There are 5 common forms of initialization in C++:

```
int a;         // default-initialization (no initializer)

// Traditional initialization forms:
int b = 5;     // copy-initialization (initial value after equals sign)
int c ( 6 );   // direct-initialization (initial value in parenthesis)

// Modern initialization forms (preferred):
int d { 7 };   // direct-list-initialization (initial value in braces)
int e {};      // value-initialization (empty braces)
```

You may see the above forms written with different spacing (e.g. `int b=5;` `int c(6);`, `int d{7};`, `int e{};`). Whether you use extra spaces for readability or not is a matter of personal preference.

As of C++17, copy-initialization, direct-initialization, and direct-list-initialization behave identically in most cases. We'll cover the most relevant case where they differ below.

> **Related content**
>
> We cover the rest of the differences between copy-initialization, direct-initialization, and list-initialization in lesson 14.15 – Class initialization and copy elision.

> **For advanced readers**
>
> Other forms of initialization include: - Aggregate initialization (see 13.8 – Struct aggregate initialization). - Copy-list-initialization (discussed below). - Reference initialization (see 12.3 – Lvalue references). - Static-initialization, constant-initialization, and dynamic-initialization (see 7.8 – Why (non-const) global variables are evil). - Zero-initialization (discussed below).

Default-initialization

When no initializer is provided (such as for variable `a` above), this is called **default-initialization**. In many cases, default-initialization performs no initialization, and leaves the variable with an indeterminate value (a value that is not predictable, sometimes called a "garbage value").

We'll discuss this case further in lesson (1.6 – Uninitialized variables and undefined behavior).

Copy-initialization

When an initial value is provided after an equals sign, this is called **copy-initialization**. This form of initialization was inherited from the C language.

```
int width = 5; // copy-initialization of value 5 into variable width
```

Much like copy-assignment, this copies the value on the right-hand side of the equals into the variable being created on the left-hand side. In the above snippet, variable `width` will be initialized with value `5`.

Copy-initialization had fallen out of favor in modern C++ due to being less efficient than other forms of initialization for some complex types. However, C++17 remedied the bulk of these issues, and copy-initialization is now finding new advocates. You will also find it used in older code (especially code ported from C), or by developers who simply think it looks more natural and is easier to read.

> **For advanced readers**
>
> Copy-initialization is also used whenever values are implicitly copied, such as when passing arguments to a function by value, returning from a function by value, or catching exceptions by value.

Direct-initialization

When an initial value is provided inside parenthesis, this is called **direct-initialization**.

```
int width ( 5 ); // direct initialization of value 5 into variable width
```

Direct-initialization was initially introduced to allow for more efficient initialization of complex objects (those with class types, which we'll cover in a future chapter). Just like copy-initialization, direct-initialization had fallen out of favor in modern C++, largely due to being superseded by direct-list-initialization. However, direct-list-initialization has a few quirks of its own, and so direct-initialization is once again finding use in certain cases.

> **For advanced readers**
>
> Direct-initialization is also used when values are explicitly cast to another type (e.g. via `static_cast`).

List-initialization

The modern way to initialize objects in C++ is to use a form of initialization that makes use of curly braces. This is called **list-initialization** (or **uniform initialization** or **brace initialization**).

List-initialization comes in two forms:

```
int width { 5 };    // direct-list-initialization of initial value 5 into variable width (preferred)
int height = { 6 }; // copy-list-initialization of initial value 6 into variable height (rarely used)
```

Prior to C++11, some types of initialization required using copy-initialization, and other types of initialization required using direct-initialization. Copy-initialization can be hard to differentiate from copy-assignment (because both use an `=`). And direct-initialization can be difficult to differentiate from function-related operations (because both use parentheses).

List-initialization was introduced to provide a initialization syntax that works in almost all cases, behaves consistently, and has an unambiguous syntax that makes it easy to tell where we're initializing an object.

> **Key insight**
>
> When we see curly braces, we know we're list-initializing an object.

Additionally, list-initialization also provides a way to initialize objects with a list of values rather than a single value (which is why it is called "list-initialization"). We show an example of this in lesson 16.2 – Introduction to std::vector and list constructors.

List-initialization disallows narrowing conversions

One of the primary benefits of list-initialization for new C++ programmers is that "narrowing conversions" are disallowed. This means that if you try to list-initialize a variable using a value that the variable can not safely hold, the compiler is required to produce a diagnostic (compilation error or warning) to notify you. For example:

```
int main()
{
    // An integer can only hold non-fractional values.
    // Initializing an int with fractional value 4.5 requires the compiler to convert 4.5 to a value an int hold.
    // Such a conversion is a narrowing conversion, since the fractional part of the value will be lost.

    int w1 { 4.5 }; // compile error: list-init does not allow narrowing conversion

    int w2 = 4.5;   // compiles: w2 copy-initialized to value 4
    int w3 (4.5);   // compiles: w3 direct-initialized to value 4

    return 0;
}
```

On line 7 of the above program, we're using a value (`4.5`) with a fractional component (`.5`) to list-initialize an integer variable (which can only hold non-fractional values). Because this is a narrowing conversion, the compiler is required to generate a diagnostic in such cases.

Copy-initialization (line 9) and direct-initialization (line 10) both silently drop the `.5` and initialize the variable with the value `4` (which probably isn't what we want). Your compiler may warn you about this (since losing data is rarely desired), but it also may not.

Note that this restriction on narrowing conversions only applies to the list-initialization, not to any subsequent assignments to the variable:

```
int main()
{
    int w1 { 4.5 }; // compile error: list-init does not allow narrowing conversion of 4.5 to 4

    w1 = 4.5;       // okay: copy-assignment allows narrowing conversion of 4.5 to 4

    return 0;
}
```

Value-initialization and zero-initialization

When a variable is initialized using an empty set of braces, a special form of list-initialization called **value-initialization** takes place. In most cases, value-initialization will implicitly initialize the variable to zero (or whatever value is closest to zero for a given type). In cases where zeroing occurs, this is called **zero-initialization**.

```
int width {}; // value-initialization / zero-initialization to value 0
```

> **For advanced readers**
>
> For class types, value-initialization (and default-initialization) may instead initialize the object to predefined default values, which may be non-zero.

List-initialization is the preferred form of initialization in modern C++

List-initialization (including value-initialization) is generally preferred over the other initialization forms because it works in most cases (and is therefore most consistent), it disallows narrowing conversions (which we normally don't want), and it supports initialization with a list of values (something we'll cover in a future lesson).

> **Best practice**

> Prefer direct-list-initialization or value-initialization to initialize your variables.

> **Author's note**
>
> Bjarne Stroustrup (creator of C++) and Herb Sutter (C++ expert) also recommend using list-initialization to initialize your variables.

In modern C++, there are some cases where list-initialization does not work as expected. We cover one such case in lesson 16.2 – Introduction to std::vector and list constructors. Because of such quirks, some experienced developers now advocate for using a mix of copy, direct, and list-initialization, depending on the circumstance. Once you are familiar enough with the language to understand the nuances of each initialization type and the reasoning behind such recommendations, you can evaluate on your own whether you find these arguments persuasive.

Q: When should I initialize with `{ 0 }` vs `{}`?

Use direct-list-initialization when you're actually using the initial value:

```
int x { 0 };    // direct-list-initialization with initial value 0
std::cout << x; // we're using that 0 value here
```

Use value-initialization when the object's value is temporary and will be replaced:

```
int x {};      // value initialization
std::cin >> x; // we're immediately replacing that value so an explicit 0 would be meaningless
```

Initialize your variables

Initialize your variables upon creation. You may eventually find cases where you want to ignore this advice for a specific reason (e.g. a performance critical section of code that uses a lot of variables), and that's okay, as long as the choice is made deliberately.

> **Related content**
>
> For more discussion on this topic, Bjarne Stroustrup (creator of C++) and Herb Sutter (C++ expert) make this recommendation themselves here.
>
> We explore what happens if you try to use a variable that doesn't have a well-defined value in lesson 1.6 – Uninitialized variables and undefined behavior.

> **Best practice**
>
> Initialize your variables upon creation.

Instantiation

The term **instantiation** is a fancy word that means a variable has been created (allocated) and initialized (this includes default initialization). An instantiated object is sometimes called an **instance**. Most often, this term is applied to class type objects, but it is occasionally applied to objects of other types as well.

Initializing multiple variables

In the last section, we noted that it is possible to define multiple variables *of the same type* in a single statement by separating the names with a comma:

```
int a, b; // create variables a and b, but do not initialize them
```

We also noted that best practice is to avoid this syntax altogether. However, since you may encounter other code that uses this style, it's still useful to talk a little bit more about it, if for no other reason than to reinforce some of the reasons you should be avoiding it.

You can initialize multiple variables defined on the same line:

```
int a = 5, b = 6;          // copy-initialization
int c ( 7 ), d ( 8 );      // direct-initialization
int e { 9 }, f { 10 };     // direct-list-initialization
int i {}, j {};            // value-initialization
```

Unfortunately, there's a common pitfall here that can occur when the programmer mistakenly tries to initialize both variables by using one initialization statement:

```
int a, b = 5;     // wrong: a is not initialized to 5!
int a = 5, b = 5; // correct: a and b are initialized to 5
```

In the top statement, variable `a` will be left uninitialized, and the compiler may or may not complain. If it doesn't, this is a great way to have your program intermittently crash or produce sporadic results. We'll talk more about what happens if you use uninitialized variables shortly.

The best way to remember that this is wrong is to note that each variable can only be initialized by its own initializer:

```
int a = 4, b = 5; // correct: a and b both have initializers
int a, b = 5;     // wrong: a doesn't have its own initializer
```

Unused initialized variables warnings

Modern compilers will typically generate warnings if a variable is initialized but not used (since this is rarely desirable). And if "treat warnings as errors" is enabled, these warnings will be promoted to errors and cause the compilation to fail.

Consider the following innocent looking program:

```
int main()
{
    int x { 5 }; // variable x defined

    // but not used anywhere

    return 0;
}
```

When compiling this with GCC and "treat warnings as errors" on, the following error is generated:

```
prog.cc: In function 'int main()':
prog.cc:3:9: error: unused variable 'x' [-Werror=unused-variable]
```

and the program fails to compile.

There are a few easy ways to fix this.

1. If the variable really is unused and you don't need it

Whitespace and Basic Formatting

Whitespace is a term that refers to characters that are used for formatting purposes. In C++, this refers primarily to spaces, tabs, and newlines. Whitespace in C++ is generally used for 3 things: separating certain language elements, inside text, and for formatting code.

Some language elements must be whitespace-separated

The syntax of the language requires that some elements are separated by whitespace. This mostly occurs when two keywords or identifiers must be placed consecutively, so the compiler can tell them apart.

For example, a variable declaration must be whitespace separated:

```
int x; // int and x must be whitespace separated
```

If we typed `intx` instead, the compiler would interpret this as an identifier, and then complain it doesn't know what identifier `intx` is.

As another example, a function's return type and name must be whitespace separated:

```
int main(); // int and main must be whitespace separated
```

When whitespace is required as a separator, the compiler doesn't care how much whitespace is used, as long as some exists.

The following variable definitions are all valid:

```
int x;
int                y;
            int
z;
```

In certain cases, newlines are used as a separator. Single-line comments are terminated by a newline.

As an example, doing something like this will get you in trouble:

```
std::cout << "Hello world!"; // This is part of the comment and
this is not part of the comment
```

Preprocessor directives (e.g. `#include <iostream>`) must be placed on separate lines:

```
#include <iostream>
#include <string>
```

Quoted text takes the amount of whitespace literally

Inside quoted text, the amount of whitespace is taken literally.

```
std::cout << "Hello world!";
```

is different than:

```
std::cout << "Hello          world!";
```

Newlines are not allowed in quoted text:

```
std::cout << "Hello
     world!"; // Not allowed!
```

Quoted text separated by nothing but whitespace (spaces, tabs, or newlines) will be concatenated:

```
std::cout << "Hello "
     "world!"; // prints "Hello world!"
```

Using whitespace to format code

Whitespace is otherwise generally ignored. This means we can use whitespace wherever we like to format our code in order to make it easier to read.

For example, the following is pretty hard to read:

```
#include <iostream>
int main(){std::cout<<"Hello world";return 0;}
```

The following is better (but still pretty dense):

```
#include <iostream>
int main() {
std::cout << "Hello world";
return 0;
}
```

And the following is even better:

```
#include <iostream>

int main()
{
    std::cout << "Hello world";

    return 0;
}
```

Statements may be split over multiple lines if desired:

```
#include <iostream>

int main()
{
    std::cout
        << "Hello world"; // works fine
    return 0;
}
```

This can be useful for particularly long statements.

Basic formatting

Unlike some other languages, C++ does not enforce any kind of formatting restrictions on the programmer. For this reason, we say that C++ is a whitespace-independent language.

This is a mixed blessing. On one hand, it's nice to have the freedom to do whatever you like. On the other hand, many different methods of formatting C++ programs have been developed throughout the years, and you will find (sometimes significant and distracting) disagreement on which ones are best. Our basic rule of thumb is that the best styles are the ones that produce the most readable code, and provide the most consistency.

Here are our recommendations for basic formatting:

1. It's fine to use either tabs or spaces for indentation (most IDEs have a setting where you can convert a tab press into the appropriate number of spaces). Developers who prefer spaces tend to do so because it ensures that code is precisely aligned as intended regardless of which editor or settings are used. Proponents of using tabs wonder why you wouldn't use the character designed to do indentation for indentation, especially as you can set the width to whatever your personal preference is. There's no right answer here – and debating it is like arguing whether cake or pie is better. It ultimately comes down to personal preference.

   Either way, we recommend you set your tabs to 4 spaces worth of indentation. Some IDEs default to 3 spaces of indentation, which is fine too.

2. There are two conventional styles for function braces.

   Many developers prefer putting the opening curly brace on the same line as the statement:

   ```
   int main() {
       // statements here
   }
   ```

   The justification for this is that it reduces the amount of vertical whitespace (as you aren't devoting an entire line to an opening curly brace), so you can fit more code on a screen. This enhances code comprehension, as you don't need to scroll as much to understand what the code is doing.

   However, in this tutorial series, we'll use the common alternative, where the opening brace appears on its own line:

   ```
   int main()
   {
       // statements here
   }
   ```

   This enhances readability, and is less error prone since your brace pairs should always be indented at the same level. If you get a compiler error due to a brace mismatch, it's very easy to see where.

3. Each statement within curly braces should start one tab in from the opening brace of the function it belongs to. For example:

   ```
   int main()
   {
       std::cout << "Hello world!\n"; // tabbed in one tab (4 spaces)
       std::cout << "Nice to meet you.\n"; // tabbed in one tab (4 spaces)
   }
   ```

4. Lines should not be too long. Typically, 80 characters has been the de facto standard for the maximum length a line should be. If a line is going to be longer, it should be split (at a reasonable spot) into multiple lines. This can be done by indenting each subsequent line with an extra tab, or if the lines are similar, by aligning it with the line above (whichever is easier to read).

```
int main()
{
    std::cout << "This is a really, really, really, really, really, really, really, "
        "really long line\n"; // one extra indentation for continuation line

    std::cout << "This is another really, really, really, really, really, really, really, "
                 "really long line\n"; // text aligned with the previous line for continuation line

    std::cout << "This one is short\n";
}
```

This makes your lines easier to read. On modern wide-screen monitors, it also allows you to place two windows with similar code side by side and compare them more easily.

> **Best practice**
>
> Consider keeping your lines to 80 chars or less in length.

> **Tip**
>
> Many editors have a built-in feature (or plugin/extension) that will show a line (called a "column guide") at a given column (e.g. at 80 characters), so you can easily see when your lines are getting too long. To see if your editor supports this, do a search on your editor's name + "Column guide".

5. If a long line is split with an operator (eg. `<<` or `+`), the operator should be placed at the beginning of the next line, not the end of the current line

```
std::cout << 3 + 4
        + 5 + 6
        * 7 * 8;
```

This helps make it clearer that subsequent lines are continuations of the previous lines, and allows you to align the operators on the left, which makes for easier reading.

6. Use whitespace to make your code easier to read by aligning values or comments or adding spacing between blocks of code.

Harder to read:

```
cost = 57;
pricePerItem = 24;
value = 5;
numberOfItems = 17;
```

Easier to read:

```
cost          = 57;
pricePerItem  = 24;
value         = 5;
numberOfItems = 17;
```

Harder to read:

```
std::cout << "Hello world!\n"; // cout lives in the iostream library
std::cout << "It is very nice to meet you!\n"; // these comments make the code hard to read
std::cout << "Yeah!\n"; // especially when lines are different lengths
```

Easier to read:

```
std::cout << "Hello world!\n";                  // cout lives in the iostream library
std::cout << "It is very nice to meet you!\n";  // these comments are easier to read
std::cout << "Yeah!\n";                         // especially when all lined up
```

Harder to read:

```
// cout lives in the iostream library
std::cout << "Hello world!\n";
// these comments make the code hard to read
std::cout << "It is very nice to meet you!\n";
// especially when all bunched together
std::cout << "Yeah!\n";
```

Easier to read:

```
// cout lives in the iostream library
std::cout << "Hello world!\n";

// these comments are easier to read
std::cout << "It is very nice to meet you!\n";

// when separated by whitespace
std::cout << "Yeah!\n";
```

We will follow these conventions throughout this tutorial, and they will become second nature to you. As we introduce new topics to you, we will introduce new style recommendations to go with those features.

Ultimately, C++ gives you the power to choose whichever style you are most comfortable with, or think is best. However, we highly recommend you utilize the same style that we use for our examples. It has been battle tested by thousands of programmers over billions of lines of code, and is optimized for success.

One exception: If you are working in someone else's code base, adopt their styles. It's better to favor consistency than your preferences.

> **Best practice**
>
> When working in an existing project, be consistent with whatever style has already been adopted.

Automatic formatting

Most modern IDEs will help you format your code as you type it in (e.g. when you create a function, the IDE will automatically indent the statements inside the function body).

However, as you add or remove code, or change the IDE's default formatting, or paste in a block of code that has different formatting, the formatting can get messed up. Fixing the formatting for part or all of a file can be a headache. Fortunately, modern IDEs typically contain an automatic formatting feature that will reformat either a selection (highlighted with your mouse) or an entire file.

For Visual Studio users

In Visual Studio, the automatic formatting options can be found under *Edit > Advanced > Format Document* and *Edit > Advanced > Format*

Selection.

For Code::Blocks users

In Code::Blocks, the automatic formatting options can be found under *Right mouse click > Format use AStyle*.

For easier access, we recommend adding a keyboard shortcut to auto-format the active file.

There are also external tools that can be used to automatically format code. clang-format is a popular one.

> **Best practice**
>
> Using the automatic formatting feature is highly recommended to keep your code's formatting style consistent.

Style guides

A **style guide** is a concise, opinionated document containing (sometimes arbitrary) programming conventions, formatting guidelines, and best practices. The goal of a style guide is to ensure that all developers on a project are programming in a consistent manner.

Some commonly referenced C++ style guides include:

- C++ Core Guidelines, maintained by Bjarne Stroustrup and Herb Sutter.
- Google.
- LLVM
- GCC/GNU

We generally favor the C++ Core Guidelines, as they are up to date and widely applicable.